JAVASCRIPT — KIẾN THỨC & BÀI TẬP TỰ HỌC

JavaScript cho người học React và NestJS

Kiến thức nền tảng từ biến đến event loop, kèm bài tập thực hành và đáp án giải thích chi tiết ở phần sau

Gồm 75 bài tập thuộc 9 chương

Mọi ví dụ và đáp án đã được chạy thật bằng Node v20.20.2: phần "Kết quả khi chạy" là output thực tế, không phải gõ tay

Thư mục thực hành: js-crash-course/bai-lam (bạn tự tạo)

Mục lục

0 Cách dùng tài liệu này — ĐỌC TRƯỚC

1 Biến, scope và hoisting — 7 bài

2 Kiểu dữ liệu, so sánh, tham chiếu và bất biến — 9 bài

3 Hàm, destructuring và method mảng — 10 bài

4 Closure — 7 bài

5 `this`, class và prototype — 8 bài

6 Promise và async/await — 10 bài

7 Event loop — 11 bài

8 Module: CommonJS và ES Module — 6 bài

9 Tổng hợp: tự viết bản thu nhỏ của React và NestJS — 7 bài

PL Bảng tra cứu và lỗi hay gặp — TRA CỨU

> **Cấu trúc:** Phần 1 gồm kiến thức và đề bài của từng chương. Phần 2 ở nửa sau là đáp án và giải thích. Bấm "Xem đáp án" để nhảy đến đúng bài, và "Về đề bài" để quay lại.

CHƯƠNG 0

Cách dùng tài liệu này

Mục tiêu

Tài liệu này lấp phần còn thiếu trong lộ trình của bạn: **React plan** chỉ có Part J ngắn, còn **NestJS plan** đi thẳng vào TypeScript (Part P). Học xong tài liệu này, bạn đọc code React và NestJS mà không còn thấy "phép màu": vì sao state không đổi, vì sao `this` bị `undefined`, vì sao Promise chưa chạy xong mà hàm đã trả về.

Bạn đã biết PHP, nên tài liệu luôn chỉ ra chỗ **JS khác PHP**. Đó là nơi người từ PHP sang hay vấp nhất.

Quy trình làm một bài (làm đúng thứ tự)

1. Đọc phần kiến thức của chương. Gõ lại các ví dụ, đừng chỉ đọc.
2. Đọc đề, **dự đoán** hoặc tự phác cách làm trên giấy.
3. Gõ code vào một file trong `js-crash-course/bai-lam/`, chạy bằng `node ten-file.js`.
4. So sánh với mục "Kết quả mong đợi". Khác thì tự debug.
5. Kẹt hơn **15 phút** mới mở đáp án. Đọc xong thì **đóng lại** và viết lại từ trí nhớ.
6. Thử "phá code": đổi `let` thành `var`, bỏ một `await`, xoá `return`, xem chuyện gì xảy ra.

Ba loại bài và độ khó

Nhãn	Bạn phải làm gì
Dự đoán	Chỉ đọc code và ghi ra output trước khi chạy. Đây là dạng hay gặp nhất khi phỏng vấn JS
Viết code	Tự viết để ra đúng "Kết quả mong đợi"
Sửa lỗi	Code đã có nhưng chạy sai, bạn tìm nguyên nhân và sửa

Độ khó: ★ dễ, ★★ vừa, ★★★ khó, cần suy nghĩ.

Chạy code

- Chạy file: `node ten-file.js`. Kiểm tra phiên bản: `node -v`. Tài liệu chạy kiểm tra trên Node 20.
- File `.js` mặc định chạy theo kiểu **CommonJS** và **sloppy mode**, trừ khi có `'use strict'` ở đầu file. File `.mjs` chạy theo kiểu **ES Module** và luôn ở strict mode. Điều này ảnh hưởng đến `this` (chương 5) và thứ tự chạy (chương 7), nên hãy dùng đúng đuôi file như trong bài.
- Đọc lỗi: dòng đầu cho biết loại lỗi (`TypeError`, `ReferenceError`...) và message, các dòng `at ...` cho biết lỗi ở file nào, dòng nào.

Học chương nào trước phần nào của lộ trình

Chương	Nên học trước	Vì sao
1–3. Biến, tham chiếu, method mảng	React Part J, 03, 04; NestJS Part P	State bất biến, render danh sách, DTO và xử lý dữ liệu
4. Closure	React Part 03, 06, 07	Stale closure, custom hook
5. `this`, class, prototype	NestJS Part P, 03	Service, DI, decorator đều dựa trên class

Chương	Nên học trước	Vì sao
6. Promise, async/await	React Part 07; NestJS Part P.6, 08	Gọi API, truy vấn database
7. Event loop	NestJS Part 18, 21; React Part 10, 11	Chạy nền, chặn server, thứ tự cập nhật
8. Module	React Part 01; NestJS Part P.10	`import/export`, CommonJS và ESM
9. Tổng hợp	Trước khi vào React và NestJS	Tự viết phiên bản thu nhỏ của `useState`, `useReducer`, middleware, DI

Checklist tự đánh giá cuối khoá

Đánh dấu khi bạn **tự làm được không cần xem**:

- Giải thích được hoisting, Temporal Dead Zone, và vì sao `var` trong vòng lặp in ra `3 3 3`.
- Biết vì sao `[] === []` là `false` và vì sao điều đó quan trọng với React.
- Cập nhật được object lồng nhau trong mảng mà không làm đổi dữ liệu gốc.
- Viết được `filter`, `map`, `reduce`, `sort` nối tiếp nhau, tự viết được `groupBy`.
- Viết được `once`, `memoize`, `debounce`, và giải thích được stale closure.
- Nói được `this` là gì trong 5 trường hợp, sửa được lỗi mất `this`.
- Phân biệt `Promise.all`, `allSettled`, `race`, `any`; không còn dùng `forEach` với `async`.
- Đoán đúng thứ tự output của bài event loop có `setTimeout`, Promise, `async/await`, `nextTick`.
- Phân biệt được CommonJS và ESM, giải thích được live binding.

PHẦN 1

KIẾN THỨC VÀ ĐỀ BÀI

Đọc kiến thức, tự làm bài rồi mới xem đáp án.

CHƯƠNG 1

Biến, scope và hoisting

Chương này trả lời hai câu hỏi: một biến **sống ở đâu**, và **từ lúc nào** dùng được nó. Trong PHP, biến trong hàm tách biệt hoàn toàn với bên ngoài, muốn dùng biến ngoài phải có `global` hoặc `use`. JS ngược lại: hàm **tự nhìn thấy** biến ở scope bao ngoài.

1. Ba cách khai báo biến

	`var`	`let`	`const`
Scope	Function	Block `{}`	Block `{}`
Hoisting	Có, giá trị ban đầu `undefined`	Có, nhưng nằm trong TDZ	Có, nhưng nằm trong TDZ
Khai báo lại cùng scope	Được	Không	Không
Gán lại	Được	Được	**Không**

Quy tắc thực tế: mặc định dùng `const`, cần gán lại thì dùng `let`, không dùng `var`. Bạn vẫn cần hiểu `var` vì code cũ và câu hỏi phỏng vấn đều có nó.

`const` chỉ cấm **gán lại biến**, không cấm sửa nội dung object mà biến trỏ tới:

```
const user = { name: 'An' };
user.name = 'Bình';       // được: sửa nội dung object
console.log(user.name);

try {
  user = {};              // không được: gán lại biến
} catch (e) {
  console.log(e.name);
}
```

KẾT QUẢ KHI CHẠY

```
Bình
TypeError
```

2. Scope và chuỗi scope

Scope là vùng code mà một biến có thể được truy cập. JS có global scope, function scope và block scope. Khi đọc một biến, JS tìm ở scope hiện tại, không có thì tìm **lên scope bao ngoài**, cứ thế đến global. Đó gọi là **scope chain**. Scope được quyết định bởi **vị trí viết code**, không phải nơi gọi hàm (gọi là lexical scope).

```
const a = 'global';

function outer() {
  const b = 'outer';

  function inner() {
    const c = 'inner';
    console.log(a, b, c);   // inner thấy cả 3 biến
  }
```

```
  inner();
  console.log(typeof c);    // outer không thấy c của inner
}

outer();
```

KẾT QUẢ KHI CHẠY

```
global outer inner
undefined
```

`var` chỉ biết đến function, nên nó **lọt ra ngoài** block `if`, `for`:

```
if (true) {
  var x = 1;
  let y = 2;
}
console.log(x);          // var lọt ra ngoài block
console.log(typeof y);   // let thì không
```

KẾT QUẢ KHI CHẠY

```
1
undefined
```

Biến trong scope con có thể **che** (shadow) biến cùng tên ở scope ngoài. Biến ngoài không bị đổi.

3. Hoisting

Trước khi chạy, JS quét qua scope và đăng ký mọi khai báo. Vì vậy có cảm giác khai báo được "kéo lên đầu". Kết quả khác nhau tuỳ loại:

- **Function declaration:** đăng ký cả thân hàm, gọi trước khi khai báo được.
- **`var`:** đăng ký tên, giá trị là `undefined` cho đến dòng gán.
- **`let`, `const`, `class`:** đăng ký tên, nhưng truy cập trước dòng khai báo sẽ ném `ReferenceError`. Khoảng từ đầu scope đến dòng khai báo gọi là **Temporal Dead Zone (TDZ)**.

```
console.log(hello());      // gọi được: function declaration
console.log(v);            // undefined: var đã đăng ký, chưa gán

try {
  console.log(l);          // TDZ
} catch (e) {
  console.log(e.name);
}

function hello() { return 'xin chào'; }
var v = 1;
let l = 2;
```

KẾT QUẢ KHI CHẠY

```
xin chào
undefined
ReferenceError
```

Function expression và **arrow function** là giá trị gán vào biến, nên chúng tuân theo luật của biến đó, không được hoist như function declaration.

4. Vòng lặp với `var` và `let`

Đây là câu phỏng vấn kinh điển. `var` tạo **một biến duy nhất** cho cả vòng lặp. `let` tạo **một biến mới cho mỗi vòng**. Callback của `setTimeout` chạy sau khi vòng lặp kết thúc (chương 7), nên nó đọc giá trị của biến lúc đó.

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var', i), 0);
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log('let', j), 0);
}
```

KẾT QUẢ KHI CHẠY

```
var 3
var 3
var 3
let 0
let 1
let 2
```

5. Biến toàn cục ngầm và strict mode

Gán vào một tên chưa khai báo: ở sloppy mode, JS âm thầm tạo biến toàn cục. Ở strict mode thì ném `ReferenceError`. File `.mjs`, code trong `class`, và code do TypeScript hay NestJS biên dịch đều chạy strict mode nên bạn thường được bảo vệ, nhưng vẫn cần biết để đọc code cũ.

> **Khác PHP:** PHP tạo biến khi gán lần đầu và mỗi hàm có scope riêng biệt. JS bắt buộc khai báo, và hàm nhìn thấy biến bên ngoài. Viết `count = 1` thiếu `let` trong JS không phải là cách khai báo biến.

Bài tập chương 1

Bài 1.1 Hoisting với `var`

Dự đoán ★☆☆

Dự đoán 3 dòng output. Dòng nào in `undefined`, dòng nào in giá trị?

CODE ĐỀ BÀI

```
console.log(name);
var name = 'An';
console.log(name);

function show() {
  console.log(age);
  var age = 20;
}
show();
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 1.1 →

Bài 1.2 Temporal Dead Zone

Dự đoán ★★☆

Dự đoán output của cả ba hàm. Chú ý: trong `c`, khối `try` có khai báo `value` riêng.

CODE ĐỀ BÀI

```
const value = 'ngoài';

function a() {
  console.log(value);
}

function b() {
  try {
    console.log(value);
  } catch (e) {
    console.log(e.name);
  }
}

function c() {
  try {
    console.log(value);
    const value = 'trong';
  } catch (e) {
    console.log(e.name);
  }
}

a();
b();
c();
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 1.2 →

Bài 1.3 Function declaration và function expression

Dự đoán ★☆☆

Dự đoán output. Mỗi lời gọi được bọc trong `try/catch` để chương trình không dừng giữa chừng.

CODE ĐỀ BÀI

```
function tryCall(label, fn) {
  try {
    console.log(label, fn());
  } catch (e) {
    console.log(label, e.name);
  }
}

tryCall('declaration', () => decl());
tryCall('var expression', () => varExpr());
tryCall('const arrow', () => constArrow());

function decl() { return 'ok'; }
var varExpr = function () { return 'ok'; };
const constArrow = () => 'ok';
```

```
tryCall('var expression sau gán', () => varExpr());
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 1.3 →

Bài 1.4 Block scope và shadowing

Dự đoán ★☆☆

Dự đoán 5 dòng output.

CODE ĐỀ BÀI

```
let x = 1;
var y = 1;

{
  let x = 2;
  var y = 2;
  console.log(x, y);
}
console.log(x, y);

function f() {
  let x = 3;
  y = 3;
  console.log(x, y);
}
f();
console.log(x, y);

for (let x = 10; x < 11; x++) {
  console.log(x);
}
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 1.4 →

Bài 1.5 `var` trong vòng lặp

Sửa lỗi ★★☆

Mảng `handlers` chứa 3 hàm, mỗi hàm phải trả về đúng chỉ số của nó, nhưng cả ba đều trả về `3`. Sửa lại, **chỉ được đổi một từ khoá**.

CODE ĐỀ BÀI

```
const handlers = [];
for (var i = 0; i < 3; i++) {
  handlers.push(() => i);
}
console.log(handlers.map((h) => h()));
```

KẾT QUẢ HIỆN TẠI (SAI)

```
[ 3, 3, 3 ]
```

KẾT QUẢ MONG ĐỢI

```
[ 0, 1, 2 ]
```

Xem đáp án bài 1.5 →

Bài 1.6 Biến toàn cục ngầm

Dự đoán ★★☆

Dự đoán output. So sánh hàm sloppy và hàm có `'use strict'`.

CODE ĐỀ BÀI

```
function sloppy() {
  total = 100;
}
sloppy();
console.log(total);
console.log(globalThis.total);

function strict() {
  'use strict';
  try {
    amount = 100;
  } catch (e) {
    console.log(e.name);
  }
}
strict();
console.log(typeof amount);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 1.6 →

Bài 1.7 Viết lại code `var` theo chuẩn hiện đại

Viết code ★★☆

Viết lại đoạn code cho sẵn: bỏ hết `var`, dùng `const` ở mọi chỗ có thể và `let` khi cần gán lại. Kết quả phải giữ nguyên.

DỮ LIỆU / KHUNG CHO SẴN

```
var prices = [100, 200, 300];
var total = 0;
for (var i = 0; i < prices.length; i++) {
  var price = prices[i];
  total = total + price;
}
var label = 'Tổng: ' + total;
console.log(label);
```

KẾT QUẢ MONG ĐỢI

```
Tổng: 600
```

Xem đáp án bài 1.7 →

CHƯƠNG 2

Kiểu dữ liệu, so sánh, tham chiếu và bất biến

Đây là chương quan trọng nhất với React. Phần lớn lỗi "đã `setState` mà giao diện không đổi" đều do chưa hiểu **tham chiếu**.

1. Kiểu dữ liệu

JS có 7 kiểu **primitive**: `string`, `number`, `bigint`, `boolean`, `undefined`, `null`, `symbol`. Mọi thứ còn lại là **object**, kể cả array và function.

```
console.log(typeof 'a', typeof 1, typeof true, typeof undefined);
console.log(typeof null);            // lỗi lịch sử của JS, không phải object thật
console.log(typeof [], typeof {});   // array cũng là object
console.log(typeof function () {});
console.log(Array.isArray([]));      // cách đúng để kiểm tra mảng
```

KẾT QUẢ KHI CHẠY

```
string number boolean undefined
object
object object
function
true
```

JS chỉ có một kiểu `number` dùng số thực dấu phẩy động, nên có sai số. Tiền nên tính bằng **số nguyên** (đồng), không dùng số lẻ.

```
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
console.log(Number.isNaN(NaN), NaN === NaN);  // NaN không bằng chính nó
```

KẾT QUẢ KHI CHẠY

```
0.30000000000000004
false
true false
```

2. So sánh và truthy/falsy

`==` ép kiểu trước khi so sánh, luật rất khó nhớ. `===` so sánh cả kiểu và giá trị. **Luôn dùng `===`.** Ngoại lệ phổ biến duy nhất là `x == null`, đúng cho cả `null` và `undefined`.

Chỉ có **8 giá trị falsy**: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Mọi thứ khác đều truthy, kể cả `[]`, `{}` và `'0'`.

> **Khác PHP:** trong PHP, mảng rỗng `[]` và chuỗi `'0'` là falsy. Trong JS, cả hai đều **truthy**. Viết `if (items)` để kiểm tra mảng rỗng là sai, phải viết `if (items.length > 0)`.

3. Ba toán tử giá trị mặc định

```
const qty = 0;
console.log(qty || 1);        // || lấy vế phải khi vế trái falsy: 0 bị thay
```

```
console.log(qty ?? 1);        // ?? chỉ thay khi null hoặc undefined: 0 được giữ

const user = { profile: null };
console.log(user.profile?.city);          // ?. gặp null/undefined thì trả undefined
console.log(user.profile?.city ?? 'N/A'); // kết hợp với ??
```

KẾT QUẢ KHI CHẠY

```
1
0
undefined
N/A
```

Trong JSX, `{count && <List />}` in ra chữ số `0` khi `count` là `0`. Nguyên nhân là `&&` trả về chính vế trái khi vế trái falsy.

4. Giá trị và tham chiếu

Primitive được lưu và sao chép theo **giá trị**. Object được truy cập qua **tham chiếu**: biến giữ "địa chỉ", gán biến chỉ sao chép địa chỉ, không sao chép object. Hai object chỉ bằng nhau khi là **cùng một object**.

```
let a = 1;
let b = a;
b = 2;
console.log(a, b);           // primitive: độc lập

const arr1 = [1, 2];
const arr2 = arr1;
arr2.push(3);
console.log(arr1, arr2);     // cùng một mảng

console.log([] === []);      // hai mảng khác nhau
console.log(arr1 === arr2);  // cùng tham chiếu
```

KẾT QUẢ KHI CHẠY

```
1 2
[ 1, 2, 3 ] [ 1, 2, 3 ]
false
true
```

> **Khác PHP:** trong PHP, `$b = $a` với mảng tạo một **bản sao** độc lập. Trong JS, `b = a` với mảng là **cùng một mảng**. Đây là khác biệt dễ gây bug nhất với người từ PHP sang.

5. Sao chép nông và sao chép sâu

Spread (`...`) chỉ sao chép **một tầng**. Object lồng bên trong vẫn dùng chung. `structuredClone` sao chép toàn bộ các tầng.

```
const original = { name: 'An', address: { city: 'Hà Nội' } };

const shallow = { ...original };
shallow.name = 'Bình';
shallow.address.city = 'Đà Nẵng';
console.log(original.name, original.address.city);
```

```
const deep = structuredClone(original);
deep.address.city = 'Huê';
console.log(original.address.city, deep.address.city);
```

KẾT QUẢ KHI CHẠY

```
An Đà Năng
Đà Năng Huê´
```

6. Bất biến (immutability): lý do React cần nó

React so sánh state cũ và mới bằng `Object.is`, tức là so sánh **tham chiếu**. Nếu bạn sửa trực tiếp object rồi set lại, tham chiếu không đổi, React có thể không render lại. Vì vậy phải **tạo object mới** cho mỗi thay đổi.

```
const jobs = [
  { id: 1, title: 'PHP', company: { name: 'A' } },
  { id: 2, title: 'React', company: { name: 'B' } },
];

// Thêm, xoá, sửa đều tạo mả̉ng mới
const added = [...jobs, { id: 3, title: 'Nest', company: { name: 'C' } }];
const removed = jobs.filter((j) => j.id !== 1);
const updated = jobs.map((j) =>
  j.id === 2 ? { ...j, company: { ...j.company, name: 'B2' } } : j
);

console.log(added.length, removed.length);
console.log(updated[1].company.name, jobs[1].company.name); // bả̂n gô´c không đổi
console.log(updated[0] === jobs[0]);  // phần tử không sửa vẫn dùng lại tham chiê´u cũ
console.log(updated[1] === jobs[1]);  // phần tử bị sửa là object mới
```

KẾT QUẢ KHI CHẠY

```
3 1
B2 B
true
false
```

Method **làm đổi mảng gốc** cần tránh với state: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`. Bản thay thế: spread, `filter`, `map`, `toSorted`, `toReversed`, `toSpliced`, `with`.

Bài tập chương 2

Bài 2.1 `typeof` Dự đoán ★☆☆

Dự đoán output của từng dòng.

CODE ĐỀ BÀI

```
console.log(typeof 42);
console.log(typeof '42');
console.log(typeof null);
console.log(typeof undefined);
console.log(typeof []);
console.log(typeof {});
console.log(typeof (() => {}));
```

```
console.log(typeof NaN);
console.log(typeof class A {});
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 2.1 →

Bài 2.2 `==` và `===`

Dự đoán ★☆☆

Dự đoán `true` hoặc `false` cho từng dòng.

CODE ĐỀ BÀI

```
console.log(1 == '1');
console.log(1 === '1');
console.log(0 == '');
console.log(0 == false);
console.log(null == undefined);
console.log(null === undefined);
console.log(null == 0);
console.log(NaN == NaN);
console.log([] == []);
console.log({} === {});
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 2.2 →

Bài 2.3 Truthy và falsy

Dự đoán ★☆☆

Dự đoán từng giá trị là truthy (`true`) hay falsy (`false`). Chú ý những giá trị khác với PHP.

CODE ĐỀ BÀI

```
const values = [0, '', '0', 'false', [], {}, null, undefined, NaN, -1];
for (const v of values) {
  console.log(JSON.stringify(v), Boolean(v));
}
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 2.3 →

Bài 2.4 `||` và `??` khi có giá trị 0

Sửa lỗi ★★☆

Hàm `withDefaults` điền giá trị mặc định khi thiếu dữ liệu, nhưng đơn hàng có `quantity: 0` và `discount: 0` đang bị thay bằng giá trị mặc định. Sửa lại để chỉ thay khi giá trị **thiếu**.

CODE ĐỀ BÀI

```
function withDefaults(item) {
  return {
    name: item.name || 'Không tên',
    quantity: item.quantity || 1,
    discount: item.discount || 10,
  };
}
```

```
console.log(withDefaults({ name: 'A', quantity: 0, discount: 0 }));
console.log(withDefaults({}));
```

KẾT QUẢ HIỆN TẠI (SAI)

```
{ name: 'A', quantity: 1, discount: 10 }
{ name: 'Không tên', quantity: 1, discount: 10 }
```

KẾT QUẢ MONG ĐỢI

```
{ name: 'A', quantity: 0, discount: 0 }
{ name: 'Không tên', quantity: 1, discount: 10 }
```

Xem đáp án bài 2.4 →

Bài 2.5 Tham số là object

Dự đoán ★★☆

Hàm nhận object có thể sửa được object gốc, nhưng không thể đổi biến bên ngoài trỏ sang object khác. Dự đoán output.

CODE ĐỀ BÀI

```
function rename(user) {
  user.name = 'Bình';
}
function replace(user) {
  user = { name: 'Chi' };
}
function bump(n) {
  n = n + 1;
}

const u = { name: 'An' };
rename(u);
console.log(u.name);
replace(u);
console.log(u.name);

let count = 1;
bump(count);
console.log(count);

const list = [1];
const alias = list;
alias.push(2);
const copy = [...list];
copy.push(3);
console.log(list, alias, copy);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 2.5 →

Bài 2.6 Bẫy sao chép nông và `JSON`

Dự đoán ★★☆

Dự đoán output của ba cách sao chép.

CODE ĐỀ BÀI

```
const source = {
  name: 'Đơn 1',
  items: [{ sku: 'A', qty: 1 }],
  createdAt: new Date('2026-01-01T00:00:00Z'),
  note: undefined,
};

const c1 = { ...source };
c1.items[0].qty = 5;
console.log(source.items[0].qty);

const c2 = JSON.parse(JSON.stringify(source));
c2.items[0].qty = 9;
console.log(source.items[0].qty);
console.log(typeof c2.createdAt);
console.log('note' in c2);

const c3 = structuredClone(source);
c3.items[0].qty = 7;
console.log(source.items[0].qty);
console.log(c3.createdAt instanceof Date);
console.log('note' in c3);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 2.6 →

Bài 2.7 Cập nhật state lồng nhau

Sửa lỗi ★★☆

Hàm `applyRename` được viết để đổi tên công ty mà **không làm đổi dữ liệu gốc**, nhưng dòng cuối cho thấy dữ liệu gốc đã bị sửa, và `same` là `true` nên React sẽ coi như không có gì thay đổi. Sửa hàm.

CODE ĐỀ BÀI

```
const state = { job: { title: 'PHP', company: { name: 'Cũ', city: 'HN' } } };

function applyRename(s, name) {
  const next = { ...s };
  next.job.company.name = name;
  return next;
}

const next = applyRename(state, 'Mới');
const same = next.job === state.job;
console.log(next.job.company.name, state.job.company.name, same);
```

KẾT QUẢ HIỆN TẠI (SAI)

```
Mới Mới true
```

KẾT QUẢ MONG ĐỢI

```
Mới Cũ false
```

Xem đáp án bài 2.7 →

Bài 2.8 Thêm, sửa, xoá mà không đổi mảng gốc

Viết code ★★☆

Với mảng `jobs` cho sẵn, viết 4 hàm thuần, **không dùng** `push`, `splice`, `sort` trực tiếp trên mảng đầu vào:

- `addJob(jobs, job)`: thêm vào cuối.
- `removeJob(jobs, id)`: xoá theo id.
- `updateSalary(jobs, id, salary)`: đổi lương của một job.
- `sortBySalaryDesc(jobs)`: trả về mảng sắp giảm dần theo lương.

DỮ LIỆU / KHUNG CHO SẴN

```
const jobs = [
  { id: 1, title: 'PHP', salary: 100 },
  { id: 2, title: 'React', salary: 300 },
  { id: 3, title: 'Nest', salary: 200 },
];
// In id của từng kết quả, và cuối cùng in mảng gốc để chứng minh nó không đổi
```

KẾT QUẢ MONG ĐỢI

```
1,2,3,4
1,3
999
2,3,1
1,2,3 200
```

Xem đáp án bài 2.8 →

Bài 2.9 Cộng tiền không sai số

Sửa lỗi ★★☆

Tổng tiền đang ra sai số vì tính bằng số lẻ. Giả sử giá lưu theo đơn vị nghìn đồng có một số lẻ. Sửa để tính trên **số nguyên đồng**, rồi in theo kiểu `1.234.500 đ`.

CODE ĐỀ BÀI

```
const prices = [10.1, 20.2, 0.3];
let total = 0;
for (const p of prices) {
  total += p;
}
console.log(total);
console.log(total === 30.6);
```

KẾT QUẢ HIỆN TẠI (SAI)

```
30.599999999999998
false
```

KẾT QUẢ MONG ĐỢI

```
30600
true
30.600 đ
```

Xem đáp án bài 2.9 →

CHƯƠNG 3

Hàm, destructuring và method mảng

React dùng `map` để render danh sách và `filter` để lọc. NestJS dùng chúng để xử lý dữ liệu từ database. Chương này cần thuộc đến mức viết không phải nghĩ.

1. Các cách viết hàm

```
function add(a, b) { return a + b; }       // declaration
const sub = function (a, b) { return a - b; }; // expression
const mul = (a, b) => a * b;               // arrow, trả về ngầm
const toObj = (id) => ({ id });            // trả object phải có ngoặc tròn
const greet = (name = 'bạn') => `Chào ${name}`; // tham số mặc định

console.log(add(2, 3), sub(5, 1), mul(2, 4), toObj(1), greet());
```

KẾT QUẢ KHI CHẠY

```
5 4 8 { id: 1 } Chào bạn
```

Hàm trong JS là **giá trị**: gán được vào biến, truyền làm tham số, trả về từ hàm khác. Hàm nhận hoặc trả về hàm gọi là **higher-order function**. Hàm truyền vào gọi là **callback**.

2. Destructuring, spread và rest

```
const user = { id: 1, name: 'An', address: { city: 'HN' }, role: 'dev' };

const { name, role: position, age = 18, address: { city }, ...rest } = user;
console.log(name, position, age, city, rest);

const [first, , third = 0, ...others] = [10, 20, 30, 40, 50];
console.log(first, third, others);

let x = 1, y = 2;
[x, y] = [y, x];          // đổi chỗ hai biến
console.log(x, y);

function sum(...nums) { return nums.reduce((s, n) => s + n, 0); } // rest
console.log(sum(1, 2, 3), Math.max(...[4, 9, 2]));                // spread
```

KẾT QUẢ KHI CHẠY

```
An dev 18 HN { id: 1 }
10 30 [ 40, 50 ]
2 1
6 9
```

Props của React chính là destructuring: `function Card({ title, salary = 0 })`. `useState` trả về mảng nên viết `const [count, setCount] = useState(0)`.

3. Bảng method mảng

Method	Dùng để	Trả về	Sửa mảng gốc?
`map`	Biến đổi từng phần tử	Mảng mới cùng độ dài	Không
`filter`	Giữ phần tử thoả điều kiện	Mảng mới	Không
`reduce`	Gộp về một giá trị	Giá trị bất kỳ	Không
`find` / `findIndex`	Tìm phần tử đầu tiên	Phần tử hoặc `undefined` / chỉ số hoặc `-1`	Không
`some` / `every`	Có phần tử nào / mọi phần tử thoả	`boolean`	Không
`includes`	Có chứa giá trị không	`boolean`	Không
`forEach`	Chạy tác vụ cho từng phần tử	`undefined`, không `break` được	Không
`flat` / `flatMap`	Làm phẳng	Mảng mới	Không
`slice`	Cắt một đoạn	Mảng mới	Không
`join`	Ghép thành chuỗi	`string`	Không
`sort` / `reverse` / `splice`	Sắp xếp / đảo / chèn, xoá	Mảng (đã bị sửa)	**Có**
`push` / `pop` / `shift` / `unshift`	Thêm, bớt ở đầu hoặc cuối	Độ dài mới / phần tử bị lấy ra	**Có**
`toSorted` / `toReversed` / `toSpliced` / `with`	Bản không sửa gốc	Mảng mới	Không

> **Khác PHP:** thứ tự là `arr.map(fn)` gọi trên chính mảng, không có chuyện lẫn lộn tham số như `array_map($fn, $arr)` và `array_filter($arr, $fn)`. Hàm `filter` của JS cũng **đánh lại chỉ số**, không giữ key như `array_filter`.

4. `reduce`: tích luỹ về một giá trị

`reduce((acc, item) => acc mới, giá trị đầu)`. Mỗi vòng nhận giá trị tích luỹ `acc`, trả về `acc` mới cho vòng sau.

```
const orders = [
  { user: 'An', total: 100 },
  { user: 'Bình', total: 200 },
  { user: 'An', total: 50 },
];

const sum = orders.reduce((acc, o) => acc + o.total, 0);

const byUser = orders.reduce((acc, o) => {
  acc[o.user] = (acc[o.user] ?? 0) + o.total;
  return acc;
}, {});
```

```
console.log(sum);
console.log(byUser);
```

KẾT QUẢ KHI CHẠY

```
350
{ An: 150, 'Bình': 200 }
```

Node 21 trở lên có sẵn `Object.groupBy(list, fn)`. Bạn vẫn nên tự viết được bằng `reduce`, vì đây là câu hay bị hỏi.

5. Bẫy của `sort`

`sort()` không có hàm so sánh sẽ đổi phần tử thành **chuỗi** rồi so sánh, nên số sắp sai. Hàm so sánh trả về số âm thì `a` đứng trước, số dương thì `b` đứng trước.

```
const nums = [10, 9, 1, 100];
console.log([...nums].sort());
console.log([...nums].sort((a, b) => a - b));
console.log(['ô', 'a', 'z', 'đ'].sort((a, b) => a.localeCompare(b, 'vi')));
```

KẾT QUẢ KHI CHẠY

```
[ 1, 10, 100, 9 ]
[ 1, 9, 10, 100 ]
[ 'a', 'đ', 'ô', 'z' ]
```

6. Object và mảng qua lại

`Object.keys`, `Object.values`, `Object.entries` đổi object thành mảng, `Object.fromEntries` đổi ngược lại. Nhờ đó dùng được `map`, `filter` trên object.

```
const prices = { a: 100, b: 200, c: 50 };
const expensive = Object.fromEntries(
  Object.entries(prices).filter(([, price]) => price >= 100)
);
console.log(expensive);
```

KẾT QUẢ KHI CHẠY

```
{ a: 100, b: 200 }
```

Bài tập chương 3

Bài 3.1 `sort` mặc định

Dự đoán ★☆☆

Dự đoán output.

CODE ĐỀ BÀI

```
const a = [3, 20, 100];
console.log(a.sort());
console.log(a);
```

```
const b = [3, 20, 100];
const sorted = b.toSorted((x, y) => y - x);
console.log(sorted, b);

const c = [1, 2, 3];
const r = c.reverse();
console.log(r === c, c);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 3.1 →

Bài 3.2 `map` với `parseInt`

Dự đoán ★★☆

Câu hỏi phỏng vấn nổi tiếng. Dự đoán 3 dòng output.

CODE ĐỀ BÀI

```
console.log(['1', '2', '3'].map(parseInt));
console.log(['1', '2', '3'].map(Number));
console.log(['1', '2', '3'].map((s) => parseInt(s, 10)));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 3.2 →

Bài 3.3 Destructuring tham số

Viết code ★☆☆

Viết hàm `formatJob` nhận một object job và trả về chuỗi như kết quả bên dưới, dùng **destructuring ngay ở tham số**. Thiếu `salary` thì mặc định là `0`, thiếu `company.city` thì in `Chưa rõ`.

DỮ LIỆU / KHUNG CHO SẴN

```
const a = { title: 'PHP Dev', salary: 15, company: { name: 'NKK', city: 'Hà Nội' } };
const b = { title: 'React Dev', company: { name: 'ABC' } };
```

KẾT QUẢ MONG ĐỢI

```
PHP Dev - 15 triệu - NKK (Hà Nội)
React Dev - 0 triệu - ABC (Chưa rõ)
```

Xem đáp án bài 3.3 →

Bài 3.4 Lọc, lấy trường, sắp xếp

Viết code ★☆☆

Từ `jobs`, lấy `title` của các job có `salary > 100`, sắp A → Z, **không làm đổi mảng gốc**. In kết quả, rồi in `title` của mảng gốc theo thứ tự cũ để chứng minh nó không đổi.

DỮ LIỆU / KHUNG CHO SẴN

```
const jobs = [
  { title: 'Vue', salary: 90 },
  { title: 'React', salary: 300 },
  { title: 'Nest', salary: 200 },
  { title: 'Laravel', salary: 150 },
  { title: 'jQuery', salary: 50 },
];
```

KẾT QUẢ MONG ĐỢI

```
[ 'Laravel', 'Nest', 'React' ]
[ 'Vue', 'React', 'Nest', 'Laravel', 'jQuery' ]
```

Xem đáp án bài 3.4 →

Bài 3.5 **Tự viết `groupBy` bằng `reduce`** Viết code ★★☆

Viết `groupBy(list, keyFn)` trả về object nhóm các phần tử theo kết quả của `keyFn`. Dùng nó để nhóm đơn hàng theo `status`, rồi tính thêm tổng tiền mỗi nhóm.

DỮ LIỆU / KHUNG CHO SẴN

```
const orders = [
  { id: 1, status: 'paid', total: 100 },
  { id: 2, status: 'new', total: 50 },
  { id: 3, status: 'paid', total: 200 },
  { id: 4, status: 'cancel', total: 80 },
];
```

KẾT QUẢ MONG ĐỢI

```
[ 'paid', 'new', 'cancel' ]
[ 1, 3 ]
{ paid: 300, new: 50, cancel: 80 }
```

Xem đáp án bài 3.5 →

Bài 3.6 **Đếm và loại trùng** Viết code ★☆☆

Với mảng `tags` lồng nhau bên dưới:

1. Làm phẳng và loại trùng, giữ thứ tự xuất hiện đầu tiên.
2. Đếm số lần xuất hiện của mỗi tag.
3. In tag xuất hiện nhiều nhất.

DỮ LIỆU / KHUNG CHO SẴN

```
const posts = [
  { tags: ['js', 'react'] },
  { tags: ['php', 'js'] },
  { tags: ['js', 'nest', 'react'] },
];
```

KẾT QUẢ MONG ĐỢI

```
[ 'js', 'react', 'php', 'nest' ]
{ js: 3, react: 2, php: 1, nest: 1 }
js
```

Xem đáp án bài 3.6 →

Bài 3.7 **Lọc và đổi key của object** Viết code ★★☆

Từ object `body`, viết hàm `pick(obj, keys)` chỉ giữ các key cho phép, và `omit(obj, keys)` bỏ các key chỉ định. Đây chính là việc `whitelist: true` trong `ValidationPipe` của NestJS làm, và `Pick`, `Omit` trong

TypeScript mô tả ở mức kiểu.

DỮ LIỆU / KHUNG CHO SẴN

```
const body = { name: 'An', email: 'a@x.vn', isAdmin: true, password: '123' };
```

KẾT QUẢ MONG ĐỢI

```
{ name: 'An', email: 'a@x.vn' }
{ name: 'An', email: 'a@x.vn' }
true
```

Xem đáp án bài 3.7 →

Bài 3.8 `sort` làm hỏng dữ liệu gốc

Sửa lỗi ★★☆

Hàm `topJobs` lấy 2 job lương cao nhất, nhưng sau khi gọi, thứ tự của mảng gốc cũng bị đổi. Trong React, lỗi này làm danh sách hiển thị đổi thứ tự một cách bí ẩn. Sửa lại.

CODE ĐỀ BÀI

```
const jobs = [
  { title: 'A', salary: 100 },
  { title: 'B', salary: 300 },
  { title: 'C', salary: 200 },
];

function topJobs(list, n) {
  return list.sort((a, b) => b.salary - a.salary).slice(0, n);
}

console.log(topJobs(jobs, 2).map((j) => j.title));
console.log(jobs.map((j) => j.title));
```

KẾT QUẢ HIỆN TẠI (SAI)

```
[ 'B', 'C' ]
[ 'B', 'C', 'A' ]
```

KẾT QUẢ MONG ĐỢI

```
[ 'B', 'C' ]
[ 'A', 'B', 'C' ]
```

Xem đáp án bài 3.8 →

Bài 3.9 Phân trang và chia lô

Viết code ★★☆

Viết hai hàm:

- `paginate(list, page, perPage)` trả về `{ data, page, perPage, total, lastPage }`, `page` bắt đầu từ 1.
- `chunk(list, size)` chia mảng thành các lô, dùng khi chèn database theo từng lô.

DỮ LIỆU / KHUNG CHO SẴN

```
const items = [1, 2, 3, 4, 5, 6, 7];
```

KẾT QUẢ MONG ĐỢI

```
{ data: [ 4, 5, 6 ], page: 2, perPage: 3, total: 7, lastPage: 3 }
[]
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7 ] ]
```

Xem đáp án bài 3.9 →

Bài 3.10 **Tự viết `map`, `filter`, `reduce`** Viết code ★★★

Viết `myMap`, `myFilter`, `myReduce` bằng vòng lặp `for`, không dùng method có sẵn. Callback phải nhận đủ `(value, index, array)`. `myReduce` không có giá trị đầu thì dùng phần tử đầu tiên, mảng rỗng và không có giá trị đầu thì ném `TypeError`.

KẾT QUẢ MONG ĐỢI

```
[ 0, 2, 6 ]
[ 2, 4 ]
6
10
TypeError
```

Xem đáp án bài 3.10 →

CHƯƠNG 4

Closure

Closure là khái niệm khó nhất với người mới, nhưng cũng là nền tảng của toàn bộ React Hooks. Hiểu closure thì stale state, dependency của `useEffect`, và `useCallback` đều trở nên dễ hiểu.

1. Closure là gì

Closure là một hàm cùng với các biến bên ngoài mà nó nhớ được từ nơi nó được tạo ra. Biến đó vẫn sống sau khi hàm bao ngoài đã chạy xong, miễn là còn hàm bên trong dùng đến.

```
function createCounter() {
  let count = 0;             // biến này không bị xoá khi createCounter kết thúc
  return () => {
    count++;
    return count;
  };
}

const a = createCounter();
const b = createCounter();   // mỗi lần gọi tạo một count riêng

console.log(a(), a(), a());
console.log(b());
```

KẾT QUẢ KHI CHẠY

```
1 2 3
1
```

Mỗi lần `createCounter` chạy, một môi trường biến mới được tạo. `a` và `b` không dùng chung `count`.

> **So với PHP:** closure PHP phải khai báo rõ bằng `use ($count)` và mặc định sao chép giá trị. Closure JS tự nhìn thấy biến ngoài và giữ **tham chiếu đến chính biến đó**, nên nó luôn đọc được giá trị mới nhất của biến, và ghi được vào biến. Giống `use (&$count)` trong PHP.

2. Closure giữ biến, không giữ giá trị

```
let name = 'An';
const greet = () => `Chào ${name}`;

name = 'Bình';
console.log(greet());    // đọc giá trị hiện tại của biến
```

KẾT QUẢ KHI CHẠY

```
Chào Bình
```

3. Ứng dụng thường gặp

Biến riêng tư: bên ngoài không thể truy cập trực tiếp, chỉ đi qua các hàm được trả về.

```
function createWallet(initial) {
  let balance = initial;
  return {
    deposit(n) { balance += n; },
    getBalance() { return balance; },
  };
}

const w = createWallet(100);
w.deposit(50);
console.log(w.getBalance(), w.balance);
```

KẾT QUẢ KHI CHẠY

```
150 undefined
```

Hàm factory: tạo hàm đã cấu hình sẵn, gọi là partial application hoặc currying.

```
const withTax = (rate) => (price) => Math.round(price * (1 + rate));
const vat10 = withTax(0.1);
const vat8 = withTax(0.08);
console.log(vat10(1000), vat8(1000));
```

KẾT QUẢ KHI CHẠY

```
1100 1080
```

Các ứng dụng khác sẽ làm trong bài tập: `once`, `memoize`, `debounce`, store.

4. Stale closure và React

Một lần render trong React là **một lần gọi hàm component**. Mỗi lần gọi có biến `count` riêng, không đổi trong suốt lần render đó. Hàm tạo trong lần render nào thì nhớ `count` của lần render đó.

Nếu hàm đó chạy muộn hơn (trong `setTimeout`, `setInterval`, `useEffect`), nó đọc giá trị **cũ**. Đó là **stale closure**. Có thể mô phỏng không cần React:

```
let state = 0;

function render() {
  const count = state;                       // snapshot của lần render này
  const logLater = () => console.log('count =', count);
  return { count, logLater };
}

const first = render();
state = 5;                                   // state đổi, render lại
const second = render();

first.logLater();                            // vẫn nhớ giá trị của lần render đầu
second.logLater();
```

KẾT QUẢ KHI CHẠY

```
count = 0
count = 5
```

Đây chính là lý do React có dạng `setCount((c) => c + 1)`: hàm cập nhật nhận giá trị **mới nhất**, không phụ thuộc vào `count` mà closure đang giữ.

5. Lưu ý về bộ nhớ

Biến được closure giữ sẽ chưa được giải phóng khi closure còn sống. Một listener hay timer không được huỷ sẽ giữ mọi thứ nó nhớ trong bộ nhớ. Đó là lý do cần cleanup trong `useEffect`. Trên server NestJS, một process chạy rất lâu nên rò rỉ kiểu này tích luỹ dần, khác với PHP-FPM, nơi bộ nhớ được giải phóng sau mỗi request.

Bài tập chương 4

Bài 4.1 **Hai counter độc lập** Dự đoán ★☆☆

Dự đoán output. Chú ý chỗ nào tạo closure mới, chỗ nào dùng lại closure cũ.

CODE ĐỀ BÀI

```
function makeCounter(start) {
  let n = start;
  return {
    inc: () => ++n,
    dec: () => --n,
    get: () => n,
  };
}

const c1 = makeCounter(0);
const c2 = makeCounter(10);
c1.inc();
c1.inc();
c2.dec();
console.log(c1.get(), c2.get());

const alias = c1;
alias.inc();
console.log(c1.get());

const { inc } = makeCounter(100);
console.log(inc(), inc());
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 4.1 →

Bài 4.2 **Closure trong vòng lặp `var`** Dự đoán ★★☆

Dự đoán output của cả ba cách. Cách thứ hai là cách người ta sửa lỗi này trước khi có `let`.

CODE ĐỀ BÀI

```
const a = [];
for (var i = 0; i < 3; i++) {
  a.push(() => i);
}
console.log(a.map((f) => f()));

const b = [];
```

```
for (var j = 0; j < 3; j++) {
  (function (k) {
    b.push(() => k);
  })(j);
}
console.log(b.map((f) => f()));

const c = [];
for (let m = 0; m < 3; m++) {
  c.push(() => m);
}
console.log(c.map((f) => f()));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 4.2 →

Bài 4.3 `once` Viết code ★☆☆

Viết `once(fn)` trả về hàm mới chỉ chạy `fn` **đúng một lần**. Các lần gọi sau trả lại kết quả lần đầu. Hàm mới phải chuyển được mọi tham số vào `fn`.

KẾT QUẢ MONG ĐỢI

```
Khởi tạo a
42
42
```

Xem đáp án bài 4.3 →

Bài 4.4 `memoize` Viết code ★★☆

Viết `memoize(fn)` nhớ kết quả theo tham số, gọi lại với cùng tham số thì không chạy `fn` nữa. Dùng `Map` để lưu cache, và `JSON.stringify(args)` làm khoá. Đây chính là ý tưởng của `useMemo`, chỉ khác là `useMemo` chỉ nhớ lần gần nhất.

KẾT QUẢ MONG ĐỢI

```
3 3 5
Số lần chạy thật: 2
```

Xem đáp án bài 4.4 →

Bài 4.5 `debounce` Viết code ★★☆

Viết `debounce(fn, delay)`: chỉ chạy `fn` sau khi đã **ngừng gọi** đủ `delay` ms, với tham số của lần gọi cuối. Mô phỏng người dùng gõ "r", "re", "rea", rồi dừng. Câu này hay gặp cả trong phỏng vấn và trong ô tìm kiếm thực tế.

KẾT QUẢ MONG ĐỢI

```
Gọi API với: rea
Gọi API với: reac
```

Xem đáp án bài 4.5 →

Stale closure khi cập nhật state

Sửa lỗi ★★★

Bài 4.6

Đây là bug React kinh điển, được mô phỏng không cần React: `setCount` đặt state, còn `tick` được tạo một lần và nhớ `count` của lần "render" đầu, giống hàm tạo trong `useEffect(..., [])`. Sau 3 lần tick, state chỉ là `1` chứ không phải `3`. Sửa để `setCount` nhận được **hàm cập nhật**, như React.

CODE ĐỀ BÀI

```
let state = 0;
const setCount = (next) => {
  state = next;
};

function component() {
  const count = state;
  const tick = () => setCount(count + 1);
  return { tick };
}

const { tick } = component();
tick();
tick();
tick();
console.log(state);
```

KẾT QUẢ HIỆN TẠI (SAI)

```
1
```

KẾT QUẢ MONG ĐỢI

```
3
```

Xem đáp án bài 4.6 →

Bài 4.7 **Mini store kiểu Redux** Viết code ★★★

Viết `createStore(reducer, initial)` trả về `{ getState, dispatch, subscribe }`:

- `dispatch(action)`: chạy reducer để tính state mới, rồi báo cho mọi listener.
- `subscribe(fn)`: đăng ký listener, **trả về hàm huỷ đăng ký**, giống cleanup trong `useEffect`.

State phải được giữ trong closure, không truy cập trực tiếp được.

KẾT QUẢ MONG ĐỢI

```
Thay đổi: 2
Thay đổi: 5
{ count: 0 }
```

Xem đáp án bài 4.7 →

CHƯƠNG 5

`this`, class và prototype

NestJS được xây trên class: controller, service, guard đều là class có `this`. Lỗi `Cannot read properties of undefined` khi truyền `this.method` làm callback là lỗi rất thường gặp. Trong PHP, `$this` luôn là object chứa phương thức. Trong JS, `this` phụ thuộc vào **cách gọi hàm**.

1. Năm quy tắc của `this`

`this` được xác định **lúc gọi hàm**, không phải lúc định nghĩa. Riêng arrow function là ngoại lệ.

Cách gọi	`this` là gì
`obj.method()`	`obj` (object đứng trước dấu chấm)
`fn()` gọi trần	`undefined` ở strict mode, `globalThis` ở sloppy mode
`new Fn()`	Object mới được tạo
`fn.call(x)`, `fn.apply(x)`, `fn.bind(x)`	`x`
Arrow function	**Không có `this` riêng**, dùng `this` của scope bao ngoài nơi nó được viết

```
'use strict';
const user = {
  name: 'An',
  regular() { return this?.name; },
};

console.log(user.regular());       // gọi qua object

const fn = user.regular;           // lấy hàm ra, mất object
console.log(fn());

console.log(fn.call({ name: 'Bình' }));   // chỉ định this
```

KẾT QUẢ KHI CHẠY

```
An
undefined
Bình
```

2. Mất `this`: lỗi phổ biến nhất

Truyền `obj.method` làm callback chỉ truyền **hàm**, không kèm object. Khi được gọi trần, `this` không còn là `obj`.

```
class Timer {
  constructor() { this.label = 'timer'; }
  print() { return this?.label; }
}

const t = new Timer();
```

```
const callLater = (cb) => cb();

console.log(callLater(t.print));              // mất this: code trong class luôn strict
console.log(callLater(() => t.print()));      // cách 1: bọc arrow function
console.log(callLater(t.print.bind(t)));      // cách 2: bind
```

KẾT QUẢ KHI CHẠY

```
undefined
timer
timer
```

Cách thứ 3 hay gặp: khai báo phương thức bằng class field arrow function, hàm sẽ gắn với instance.

3. `call`, `apply`, `bind`

- `fn.call(ctx, a, b)`: gọi ngay, truyền tham số lần lượt.
- `fn.apply(ctx, [a, b])`: gọi ngay, truyền tham số bằng mảng.
- `fn.bind(ctx, a)`: **không gọi**, trả về hàm mới có `this` cố định và có thể gắn trước một số tham số.

4. Class

```
class Account {
  #balance = 0;              // private thật sự, chỉ dùng được trong class
  static count = 0;          // thuộc về class, không thuộc instance

  constructor(owner) {
    this.owner = owner;
    Account.count++;
  }

  deposit(n) {
    this.#balance += n;
    return this;             // trả về this để gọi nối tiếp
  }

  get balance() { return this.#balance; }   // getter dùng như thuộc tính
}

class SavingAccount extends Account {
  constructor(owner, rate) {
    super(owner);            // bắt buộc gọi super trước khi dùng this
    this.rate = rate;
  }
  addInterest() { return this.deposit(this.balance * this.rate); }
}

const s = new SavingAccount('An', 0.1);
s.deposit(1000).addInterest();
console.log(s.owner, s.balance, Account.count);
console.log(s instanceof Account);
```

KẾT QUẢ KHI CHẠY

```
An 1100 1
true
```

`private` của TypeScript chỉ kiểm tra lúc biên dịch, khi chạy vẫn truy cập được. `#field` của JS mới là riêng tư thật sự lúc chạy. Hai thứ này khác nhau, bạn sẽ gặp `private` trong NestJS.

5. Prototype

JS không có class "thật" như PHP. Mỗi object có liên kết ẩn đến một object khác gọi là **prototype**. Khi đọc một thuộc tính không có trên object, JS tìm lên prototype, rồi prototype của prototype, đến khi gặp `null`. Đây là **prototype chain**. Cú pháp `class` chỉ là cách viết gọn trên cơ chế này: phương thức nằm trên `Class.prototype`, được mọi instance dùng chung.

```
class Dog {
  constructor(name) { this.name = name; }
  bark() { return `${this.name}: gâu`; }
}

const d = new Dog('Vàng');
console.log(Object.getPrototypeOf(d) === Dog.prototype);
console.log(Object.hasOwn(d, 'name'), Object.hasOwn(d, 'bark'));
console.log(typeof Dog);

const d2 = new Dog('Đen');
console.log(d.bark === d2.bark);   // phương thức dùng chung
```

KẾT QUẢ KHI CHẠY

```
true
true false
function
true
```

Bài tập chương 5

Bài 5.1 Mất `this` khi lấy phương thức ra

Dự đoán ★☆☆

Dự đoán output. File có `'use strict'`, nên `this` khi gọi trần là `undefined`.

CODE ĐỀ BÀI

```
'use strict';

const cart = {
  name: 'Giỏ A',
  getName() {
    return this === undefined ? 'this là undefined' : this.name;
  },
};

console.log(cart.getName());

const get = cart.getName;
console.log(get());

const other = { name: 'Giỏ B', getName: cart.getName };
console.log(other.getName());
```

```
console.log((0, cart.getName)());
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 5.1 →

Bài 5.2 Arrow function và `this`

Dự đoán ★★☆

Dự đoán output. Hãy xác định arrow function được **viết** ở đâu.

CODE ĐỀ BÀI

```
'use strict';

const obj = {
  name: 'obj',
  arrow: () => (this === module.exports ? 'this của file' : 'khác'),
  regular() {
    const inner = () => this.name;
    return inner();
  },
  broken() {
    function inner() {
      return this === undefined ? 'undefined' : this.name;
    }
    return inner();
  },
};

console.log(obj.arrow());
console.log(obj.regular());
console.log(obj.broken());
console.log(obj.arrow.call({ name: 'x' }));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 5.2 →

Bài 5.3 Mất `this` trong callback

Sửa lỗi ★★☆

Giống một service NestJS có phương thức được truyền làm callback vào `map`. Chương trình đang bắt được `TypeError`. Sửa để in đúng giá sau khi giảm, **giữ nguyên** class và chỉ sửa dòng gọi `map`.

CODE ĐỀ BÀI

```
class PriceService {
  constructor() {
    this.discount = 0.1;
  }
  apply(price) {
    return price * (1 - this.discount);
  }
}

const service = new PriceService();
try {
  console.log([100, 200].map(service.apply));
} catch (e) {
```

```
  console.log(e.name);
}
```

KẾT QUẢ HIỆN TẠI (SAI)

```
TypeError
```

KẾT QUẢ MONG ĐỢI

```
[ 90, 180 ]
```

Xem đáp án bài 5.3 →

Bài 5.4 **Dùng `call`, `apply`, `bind`** Viết code ★☆☆

Với hàm `introduce` cho sẵn, gọi nó cho object `user` bằng cả ba cách. `bind` phải gắn trước tham số `greeting`.

DỮ LIỆU / KHUNG CHO SẴN

```
function introduce(greeting, punctuation) {
  return `${greeting}, tôi là ${this.name}${punctuation}`;
}
const user = { name: 'An' };
```

KẾT QUẢ MONG ĐỢI

```
Xin chào, tôi là An!
Hello, tôi là An.
Hi, tôi là An?
Hi, tôi là An?
```

Xem đáp án bài 5.4 →

Bài 5.5 **Tự viết `bind`** Viết code ★★☆

Viết hàm `myBind(fn, ctx, ...preset)` hoạt động như `fn.bind(ctx, ...preset)`, dùng closure và `call`.

KẾT QUẢ MONG ĐỢI

```
Tổng: 15
Tổng: 30
```

Xem đáp án bài 5.5 →

Bài 5.6 **Class có private field và kế thừa** Viết code ★★☆

Viết class `Product` có:

- Thuộc tính `name` và private `#price`.
- Getter `price`. Setter `price` ném `RangeError` khi giá âm.
- Phương thức `describe()`.
- Class con `DiscountProduct` thêm `percent`, ghi đè `describe()` và dùng `super.describe()`.

KẾT QUẢ MONG ĐỢI

```
Áo: 200 (giảm 10%)
RangeError Giá không được âm
200 [ 'name', 'percent' ]
```

Xem đáp án bài 5.6 →

Bài 5.7 Prototype chain

Dự đoán ★★☆

Dự đoán output. Thuộc tính ghi vào object đứng trước, hay ghi lên prototype?

CODE ĐỀ BÀI

```
class A {
  hello() { return 'A.hello'; }
}
class B extends A {
  hi() { return 'B.hi'; }
}

const b = new B();
console.log(b.hello(), b.hi());
console.log(Object.hasOwn(b, 'hi'));
console.log(b instanceof A, b instanceof B);

b.hello = () => 'own hello';
console.log(b.hello());
console.log(new B().hello());

A.prototype.hello = () => 'patched';
console.log(new B().hello());
delete b.hello;
console.log(b.hello());
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 5.7 →

Bài 5.8 Class field arrow function và `this`

Dự đoán ★★★

Dự đoán output. So sánh phương thức thường và class field arrow function, cái nào giữ được `this` và có dùng chung giữa các instance không.

CODE ĐỀ BÀI

```
class Button {
  label = 'Lưu';
  handleA() {
    return this?.label;
  }
  handleB = () => this.label;
}

const b1 = new Button();
const b2 = new Button();

const a = b1.handleA;
const { handleB } = b1;
```

```
console.log(a());
console.log(handleB());

console.log(b1.handleA === b2.handleA);
console.log(b1.handleB === b2.handleB);
console.log(Object.hasOwn(b1, 'handleA'), Object.hasOwn(b1, 'handleB'));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 5.8 →

CHƯƠNG 6

Promise và async/await

PHP chạy tuần tự: `file_get_contents` xong mới đến dòng sau. JS thì không: các thao tác I/O không chặn, kết quả đến sau. Mọi lời gọi API trong React và mọi truy vấn database trong NestJS đều là Promise.

1. Promise là gì

Promise là object đại diện cho **kết quả sẽ có trong tương lai**. Nó có 3 trạng thái: `pending` (đang chờ), `fulfilled` (thành công), `rejected` (thất bại). Đã chốt rồi thì không đổi nữa.

```
const sleep = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

sleep(10, 'xong').then((v) => console.log('then:', v));
console.log('dòng này chạy trước');
```

KẾT QUẢ KHI CHẠY

```
dòng này chạy trước
then: xong
```

2. Nối chuỗi với `then` và `catch`

`then` luôn trả về **Promise mới**. Giá trị bạn `return` trong `then` được chuyển cho `then` kế tiếp. Trả về một Promise thì chuỗi sẽ chờ nó. Lỗi được ném ra sẽ bỏ qua các `then` và nhảy đến `catch` gần nhất. Sau `catch`, chuỗi lại tiếp tục bình thường.

```
Promise.resolve(1)
  .then((x) => x + 1)
  .then((x) => { throw new Error('hỏng ở ' + x); })
  .then((x) => console.log('bị bỏ qua', x))
  .catch((e) => { console.log(e.message); return 100; })
  .then((x) => console.log('tiếp tục với', x))
  .finally(() => console.log('luôn chạy'));
```

KẾT QUẢ KHI CHẠY

```
hỏng ở 2
tiếp tục với 100
luôn chạy
```

3. `async` và `await`

- Hàm `async` **luôn trả về Promise**. `return x` tương đương resolve với `x`, `throw` tương đương reject.
- `await p` tạm dừng **hàm async đó** cho đến khi `p` chốt. Hàm dừng, nhưng chương trình vẫn chạy tiếp việc khác.
- Lỗi của Promise thành exception tại dòng `await`, bắt bằng `try/catch`.

```
const getUser = async (id) => {
  if (id < 0) throw new Error('id không hợp lệ');
  return { id, name: 'An' };
```

```
};

async function main() {
  console.log(getUser(1) instanceof Promise);
  console.log(await getUser(1));
  try {
    await getUser(-1);
  } catch (e) {
    console.log('Bắt được:', e.message);
  }
}

main();
```

KẾT QUẢ KHI CHẠY

```
true
{ id: 1, name: 'An' }
Bắt được: id không hợp lệ
```

4. Chạy tuần tự và chạy song song

```
const task = (name, ms) => new Promise((r) => setTimeout(() => r(name), ms));

async function main() {
  let t = Date.now();
  await task('a', 100);
  await task('b', 100);
  console.log('tuần tự khoảng', Math.round((Date.now() - t) / 100) * 100, 'ms');

  t = Date.now();
  const results = await Promise.all([task('a', 100), task('b', 100)]);
  console.log('song song khoảng', Math.round((Date.now() - t) / 100) * 100, 'ms', results);
}

main();
```

KẾT QUẢ KHI CHẠY

```
tuần tự khoảng 200 ms
song song khoảng 100 ms [ 'a', 'b' ]
```

Hàm	Trả kết quả khi	Reject khi	Dùng khi
`Promise.all`	Tất cả thành công, kết quả đúng thứ tự	**Ngay khi có một cái lỗi**	Cần đủ mọi kết quả
`Promise.allSettled`	Tất cả đã xong, cả lỗi lẫn thành công	Không bao giờ	Muốn biết cái nào lỗi, vẫn lấy phần còn lại
`Promise.race`	Cái **đầu tiên chốt**, thành công hay lỗi đều được	Nếu cái đầu tiên lỗi	Đặt timeout
`Promise.any`	Cái **đầu tiên thành công**	Khi tất cả đều lỗi (`AggregateError`)	Gọi nhiều nguồn dự phòng

5. Các bẫy hay gặp

- `forEach` **không chờ** `async`**:** dùng `for...of` với `await` (tuần tự), hoặc `Promise.all(list.map(...))` (song song).
- **Quên** `await`**:** biến nhận được Promise chứ không phải dữ liệu. `if (user)` luôn đúng vì Promise là truthy.
- `return promise` **trong** `try`**:** lỗi không bị `catch` của hàm đó bắt, phải viết `return await promise`.
- `fetch` **không reject khi gặp 404 hoặc 500:** chỉ reject khi lỗi mạng. Phải tự kiểm tra `res.ok`.
- **Không truyền** `async` **vào** `useEffect` **trực tiếp:** hàm `async` trả về Promise, trong khi React chờ một hàm cleanup. Khai báo hàm `async` bên trong rồi gọi.
- **Promise bị bỏ rơi (floating):** gọi mà không `await` hay `.catch`. Từ Node 15, rejection không được xử lý sẽ **làm sập process**.

```
// Không chạy đoạn này: nó làm sập process
async function job() { throw new Error('boom'); }
job();   // không await, không catch -> unhandledRejection -> process thoát mã lỗi 1
```

> **Khác PHP:** một exception chưa bắt trong PHP-FPM chỉ làm hỏng request đó. Trong Node, một rejection không được xử lý có thể làm sập **cả server**, kéo theo mọi request đang chạy. NestJS bắt được lỗi trong controller của nó, nhưng không bắt được Promise bị bỏ rơi.

Bài tập chương 6

Bài 6.1 Chuỗi `then`

Dự đoán ★☆☆

Dự đoán output.

CODE ĐỀ BÀI

```
Promise.resolve(2)
  .then((x) => x * 2)
  .then((x) => {
    console.log(x);
  })
  .then((x) => {
    console.log(x);
    return Promise.resolve('p');
  })
  .then((x) => {
    console.log(x);
    return 'cuối';
  })
  .then(console.log);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 6.1 →

Bài 6.2 Luồng lỗi với `catch`

Dự đoán ★★☆

Dự đoán output. Sau khi `catch` xử lý xong, chuỗi có chạy tiếp không?

CODE ĐỀ BÀI

```
Promise.reject(new Error('A'))
  .then(() => console.log('1'))
  .catch((e) => {
    console.log('catch', e.message);
    return 'phục hồi';
  })
  .then((v) => {
    console.log('2', v);
    throw new Error('B');
  })
  .catch((e) => {
    console.log('catch', e.message);
    throw new Error('C');
  })
  .then(() => console.log('3'))
  .catch((e) => console.log('catch', e.message))
  .finally(() => console.log('finally'));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 6.2 →

Bài 6.3 `async` trả về gì

Dự đoán ★☆☆

Dự đoán output và thứ tự in.

CODE ĐỀ BÀI

```
async function a() {
  return 1;
}
async function b() {
  throw new Error('lỗi b');
}
async function c() {
  const x = await a();
  return x + 1;
}

console.log(a());
console.log(a() instanceof Promise);

b().catch((e) => console.log(e.message));
c().then((v) => console.log('c =', v));

const r = c();
console.log(typeof r, r === 2);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 6.3 →

Bài 6.4 `forEach` không chờ `async`

Sửa lỗi ★★☆

Hàm `saveAll` phải lưu xong mọi bản ghi rồi mới in `Đã lưu xong`, nhưng dòng này đang in trước và `saved` còn rỗng. Sửa để lưu **song song** và in đúng thứ tự.

CODE ĐỀ BÀI

```
const saved = [];
const save = (x) =>
  new Promise((r) => setTimeout(() => {
    saved.push(x);
    r();
  }, 10));

async function saveAll(items) {
  items.forEach(async (x) => {
    await save(x);
  });
  console.log('Đã lưu xong', saved.length);
}

saveAll([1, 2, 3]).then(() => setTimeout(() => console.log('Thực tế', saved.length), 50));
```

KẾT QUẢ HIỆN TẠI (SAI)

```
Đã lưu xong 0
Thực tế 3
```

KẾT QUẢ MONG ĐỢI

```
Đã lưu xong 3
Thực tế 3
```

Xem đáp án bài 6.4 →

Bài 6.5 `return` so với `return await` trong `try`

Sửa lỗi ★★☆

Hàm `load` muốn bắt lỗi và trả về giá trị dự phòng, nhưng `catch` không chạy và lỗi đi thẳng ra ngoài. Chỉ cần thêm một từ.

CODE ĐỀ BÀI

```
const query = async () => {
  throw new Error('DB lỗi');
};

async function load() {
  try {
    return query();
  } catch (e) {
    console.log('Bắt được trong load');
    return 'dự phòng';
  }
}

load()
  .then((v) => console.log('Kết quả:', v))
  .catch((e) => console.log('Lỗi lọt ra ngoài:', e.message));
```

KẾT QUẢ HIỆN TẠI (SAI)

```
Lỗi lọt ra ngoài: DB lỗi
```

KẾT QUẢ MONG ĐỢI

```
Bắt được trong load
Kết quả: dự phòng
```

Xem đáp án bài 6.5 →

Bài 6.6 Lấy kết quả khi có phần bị lỗi

Viết code ★★☆

Gọi 4 "API" song song, một số sẽ lỗi. Dùng `Promise.allSettled` để in báo cáo từng cái, rồi in số thành công và danh sách lỗi. So sánh: nếu dùng `Promise.all` thì sao?

DỮ LIỆU / KHUNG CHO SẴN

```
const fetchUser = (id) =>
  id % 2 === 0
    ? Promise.resolve({ id, name: `User ${id}` })
    : Promise.reject(new Error(`Không tìm thấy ${id}`));
const ids = [1, 2, 3, 4];
```

KẾT QUẢ MONG ĐỢI

```
1 rejected Không tìm thấy 1
2 fulfilled User 2
3 rejected Không tìm thấy 3
4 fulfilled User 4
2 [ 'Không tìm thấy 1', 'Không tìm thấy 3' ]
Promise.all: Không tìm thấy 1
```

Xem đáp án bài 6.6 →

Bài 6.7 Timeout bằng `Promise.race`

Viết code ★★☆

Viết `withTimeout(promise, ms)`: nếu `promise` chưa xong sau `ms` thì reject với `Error('Timeout')`. Nhớ dọn timer để process không bị giữ lại thêm thời gian.

KẾT QUẢ MONG ĐỢI

```
nhanh
Timeout
```

Xem đáp án bài 6.7 →

Bài 6.8 `retry` có backoff

Viết code ★★☆

Viết `retry(fn, times, delay)`: gọi `fn` (hàm trả về Promise), lỗi thì đợi rồi thử lại, mỗi lần gấp đôi thời gian chờ (exponential backoff). Hết lượt thì ném lỗi cuối cùng. Kiểm tra với hàm giả lập: lỗi 2 lần rồi thành công.

KẾT QUẢ MONG ĐỢI

```
Lần 1 lỗi, chờ 10 ms
Lần 2 lỗi, chờ 20 ms
thành công
Lần 1 lỗi, chờ 10 ms
Bỏ cuộc: luôn lỗi
```

Xem đáp án bài 6.8 →

Bài 6.9 Giới hạn số việc chạy đồng thời

Viết code ★★★

`Promise.all` với 1000 request có thể làm sập server đích. Viết `mapLimit(items, limit, fn)`: chạy `fn(item)` song song, **tối đa `limit` việc cùng lúc**, kết quả đúng thứ tự đầu vào. In ra số việc đang chạy cao nhất để kiểm chứng.

KẾT QUẢ MONG ĐỢI

```
[
   2,  4,  6, 8,
  10, 12, 14
]
Đồng thời cao nhất: 3
```

Xem đáp án bài 6.9 →

Bài 6.10 Bọc `fetch` đúng cách

Sửa lỗi ★★☆

Hàm `getJob` được kỳ vọng ném lỗi khi server trả 404, nhưng `fetch` **không reject** với 404. Vì vậy code tiếp tục chạy và trả về dữ liệu lỗi như thể thành công. Sửa bằng cách kiểm tra `res.ok`. Dùng `fetch` giả lập để chạy không cần mạng.

CODE ĐỀ BÀI

```
const fakeFetch = async (url) =>
  url === '/jobs/1'
    ? { ok: true, status: 200, json: async () => ({ id: 1, title: 'PHP' }) }
    : { ok: false, status: 404, json: async () => ({ message: 'Not found' }) };

async function getJob(id) {
  const res = await fakeFetch(`/jobs/${id}`);
  return res.json();
}

async function main() {
  console.log(await getJob(1));
  try {
    console.log('Vẫn "thành công":', await getJob(2));
  } catch (e) {
    console.log('Lỗi:', e.message);
  }
}

main();
```

KẾT QUẢ HIỆN TẠI (SAI)

```
{ id: 1, title: 'PHP' }
Vẫn "thành công": { message: 'Not found' }
```

KẾT QUẢ MONG ĐỢI

```
{ id: 1, title: 'PHP' }
Lỗi: HTTP 404
```

Xem đáp án bài 6.10 →

CHƯƠNG 7

Event loop

Câu hỏi "đoán thứ tự output" gần như chắc chắn gặp khi phỏng vấn JS. Quan trọng hơn, event loop giải thích vì sao một vòng lặp nặng có thể làm cả server NestJS đứng hình.

1. Mô hình

JS chạy trên **một luồng**, có một **call stack**. Tại một thời điểm chỉ có một đoạn code chạy, và đoạn đó chạy đến khi xong mới đến đoạn khác. Các việc chờ đợi (timer, mạng, đọc file) được giao cho môi trường bên ngoài (trình duyệt, hoặc thư viện libuv của Node). Khi xong, callback được đưa vào hàng đợi:

- **Microtask queue:** callback của Promise (`then` , `catch` , `finally`), phần code sau `await` , `queueMicrotask` .
- **Macrotask queue (task queue):** `setTimeout` , `setInterval` , I/O, sự kiện giao diện, `setImmediate` trong Node.

Thứ tự luôn là:

1. Chạy hết code đồng bộ đang có trên stack.
2. Chạy **hết toàn bộ** microtask, kể cả microtask mới sinh ra trong lúc này.
3. Lấy **một** macrotask ra chạy, rồi quay lại bước 1.

```
console.log('1 đồng bộ');
setTimeout(() => console.log('4 macrotask'), 0);
Promise.resolve().then(() => console.log('3 microtask'));
console.log('2 đồng bộ');
```

KẾT QUẢ KHI CHẠY

```
1 đồng bộ
2 đồng bộ
3 microtask
4 macrotask
```

`setTimeout(fn, 0)` không có nghĩa là chạy ngay. Nó nghĩa là "chạy sớm nhất có thể, sau khi stack rỗng và microtask đã hết". Delay là thời gian **tối thiểu**, không phải chính xác.

2. `await` chia hàm làm hai

Code trước `await` đầu tiên chạy **đồng bộ** ngay khi gọi hàm. Tại `await` , hàm dừng lại và trả quyền điều khiển cho nơi gọi. Phần sau `await` trở thành một microtask.

```
async function f() {
  console.log('A: trước await, vẫn đồng bộ');
  await null;
  console.log('C: sau await, là microtask');
}

f();
console.log('B: code sau lời gọi f');
```

KẾT QUẢ KHI CHẠY

```
A: trước await, vẫn đồng bộ
B: code sau lời gọi f
C: sau await, là microtask
```

3. Thêm chi tiết trong Node

Event loop của Node đi qua các pha, các pha chính gồm: `timers` (chạy `setTimeout`, `setInterval`) → `poll` (nhận kết quả I/O) → `check` (chạy `setImmediate`) → `close`. Sau mỗi callback, Node xử lý hết `process.nextTick` rồi đến Promise microtask.

API	Chạy khi nào
`process.nextTick`	Ngay sau thao tác hiện tại, trước Promise microtask (trong CommonJS)
Promise, `queueMicrotask`	Sau code đồng bộ, trước macrotask
`setTimeout(fn, 0)`	Pha `timers`
`setImmediate`	Pha `check`, ngay sau `poll`

Ở file chính, thứ tự giữa `setTimeout(fn, 0)` và `setImmediate` **không chắc chắn**, tuỳ tốc độ máy lúc khởi động. Trong callback I/O thì `setImmediate` luôn chạy trước. Mình không đưa trường hợp không chắc chắn vào bài tập vì output có thể đổi giữa các lần chạy.

4. Vì sao Node "đơn luồng" mà vẫn phục vụ nhiều request

Khi một request `await` database, luồng JS không chờ mà đi xử lý request khác. Khi database trả lời, callback được đưa vào hàng đợi. Vì vậy Node rất hợp với công việc **nhiều I/O**.

Ngược lại, code **tốn CPU** (vòng lặp lớn, xử lý ảnh, `JSON.parse` file khổng lồ, `bcrypt` đồng bộ) không nhả luồng, nên **mọi request khác phải chờ**. Cách xử lý: chia nhỏ việc, dùng `worker_threads`, hoặc đưa ra process riêng bằng queue (NestJS Part 18).

> **Khác PHP:** PHP-FPM có nhiều worker, một request chạy lâu chỉ giữ một worker. Trong Node, một đoạn code chặn sẽ chặn **tất cả** request trong process đó. `async` không tự làm code nhanh hơn hay chạy ở luồng khác: một hàm `async` tính toán nặng vẫn chặn event loop.

5. Trong trình duyệt

Trình duyệt vẽ lại giao diện **giữa các macrotask**, sau khi microtask đã hết. Code đồng bộ chạy lâu hoặc microtask nối tiếp nhau không dứt sẽ làm giao diện đơ. React dùng các cơ chế như `useTransition` để không chặn việc phản hồi người dùng khi render nặng.

Bài tập chương 7

Bài 7.1 Thứ tự cơ bản

Dự đoán ★☆☆

Dự đoán thứ tự 6 dòng.

CODE ĐỀ BÀI

```
console.log('a');

setTimeout(() => console.log('b'), 0);

Promise.resolve().then(() => console.log('c'));

console.log('d');

Promise.resolve().then(() => console.log('e'));

setTimeout(() => console.log('f'), 0);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.1 →

Bài 7.2 Lồng microtask và macrotask

Dự đoán ★★☆

Dự đoán thứ tự. Mỗi khi một callback chạy xong, phải kiểm tra lại microtask queue.

CODE ĐỀ BÀI

```
setTimeout(() => {
  console.log('t1');
  Promise.resolve().then(() => console.log('p trong t1'));
}, 0);

setTimeout(() => console.log('t2'), 0);

Promise.resolve().then(() => {
  console.log('p1');
  setTimeout(() => console.log('t trong p1'), 0);
  Promise.resolve().then(() => console.log('p trong p1'));
});

Promise.resolve().then(() => console.log('p2'));

console.log('sync');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.2 →

Bài 7.3 `async/await` xen kẽ

Dự đoán ★★☆

Dự đoán thứ tự. Xác định phần nào của mỗi hàm chạy đồng bộ.

CODE ĐỀ BÀI

```
async function one() {
  console.log('one bắt đâù');
  await two();
  console.log('one kêt thúc');
}

async function two() {
  console.log('two');
}
```

```
console.log('script bắt đầu');
setTimeout(() => console.log('timeout'), 0);
one();
Promise.resolve().then(() => console.log('promise'));
console.log('script kết thúc');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.3 →

Bài 7.4 Đan xen hai chuỗi Promise

Dự đoán ★★★

Mỗi `then` là một microtask mới, chỉ được xếp hàng **sau khi** `then` trước nó chạy xong. Dự đoán thứ tự.

CODE ĐỀ BÀI

```
Promise.resolve()
  .then(() => console.log('A1'))
  .then(() => console.log('A2'))
  .then(() => console.log('A3'));

Promise.resolve()
  .then(() => console.log('B1'))
  .then(() => console.log('B2'));

queueMicrotask(() => console.log('C1'));
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.4 →

Bài 7.5 `nextTick` và Promise trong CommonJS

Dự đoán ★★☆

Dự đoán thứ tự trong file `.js` (CommonJS).

CODE ĐỀ BÀI

```
setTimeout(() => console.log('timeout'), 0);

Promise.resolve().then(() => console.log('promise'));

process.nextTick(() => console.log('nextTick'));

queueMicrotask(() => console.log('microtask'));

console.log('sync');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.5 →

Bài 7.6 Cùng code đó trong ES Module

Dự đoán ★★★

Vẫn đoạn code của bài trước, nhưng chạy ở file `.mjs`. Dự đoán xem thứ tự có đổi không.

CODE ĐỀ BÀI

```
setTimeout(() => console.log('timeout'), 0);
```

```
Promise.resolve().then(() => console.log('promise'));

process.nextTick(() => console.log('nextTick'));

queueMicrotask(() => console.log('microtask'));

console.log('sync');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.6 →

Bài 7.7 `setImmediate` trong callback I/O

Dự đoán ★★☆

Dự đoán thứ tự. Callback của `readFile` chạy ở pha `poll`. Pha nào đứng ngay sau đó?

CODE ĐỀ BÀI

```
const fs = require('node:fs');

fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
  Promise.resolve().then(() => console.log('promise'));
  process.nextTick(() => console.log('nextTick'));
  console.log('trong readFile');
});

console.log('sync');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.7 →

Bài 7.8 Code đồng bộ chặn timer

Dự đoán ★★☆

Timer đặt 10 ms, nhưng có vòng lặp chặn 50 ms ngay sau đó. Dự đoán thứ tự và giá trị boolean.

CODE ĐỀ BÀI

```
const start = Date.now();

setTimeout(() => {
  const waited = Date.now() - start;
  console.log('timer chạy, chờ ít nhất 50 ms:', waited >= 50);
}, 10);

while (Date.now() - start < 50) {
  // chặn luồng
}
console.log('vòng lặp xong');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.8 →

Bài 7.9 Microtask làm đói macrotask

Dự đoán ★★☆

Dự đoán thứ tự. Microtask mới sinh ra có được chạy ngay trong đợt hiện tại không?

CODE ĐỀ BÀI

```
let n = 0;

function loop() {
  n++;
  console.log('micro', n);
  if (n < 3) queueMicrotask(loop);
}

setTimeout(() => console.log('timeout'), 0);
queueMicrotask(loop);
console.log('sync');
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 7.9 →

Bài 7.10 Chia nhỏ việc nặng để không chặn

Viết code ★★★

Tính tổng từ 1 đến 5.000.000 hai cách và kiểm tra "heartbeat", một timer đại diện cho request khác, có chạy được **trước khi** việc nặng xong không:

- `sumBlocking(n)`: vòng lặp thường.
- `sumChunked(n)`: chia thành từng đoạn 100.000 số, nhả luồng bằng `setImmediate` giữa các đoạn, trả về Promise.

KẾT QUẢ MONG ĐỢI

```
Chặn: 12500002500000 - heartbeat đã chạy: false
Chia nhỏ: 12500002500000 - heartbeat đã chạy: true
```

Xem đáp án bài 7.10 →

Bài 7.11 Đơn luồng vẫn có race condition

Sửa lỗi ★★★

"JS đơn luồng nên không có race condition" là một hiểu lầm. Giữa các `await`, luồng khác có thể chen vào. Hai giao dịch nạp tiền chạy song song, nhưng kết quả mất một lần nạp (phải là `150`). Giữ nguyên `delay` mô phỏng độ trễ, chỉ sửa `deposit`.

CODE ĐỀ BÀI

```
let balance = 0;
const delay = () => new Promise((r) => setTimeout(r, 10));

async function deposit(amount) {
  const current = balance;
  await delay();
  balance = current + amount;
}

Promise.all([deposit(100), deposit(50)]).then(() => console.log(balance));
```

KẾT QUẢ HIỆN TẠI (SAI)

```
50
```

KẾT QUẢ MONG ĐỢI

```
150
```

Xem đáp án bài 7.11 →

CHƯƠNG 8

Module: CommonJS và ES Module

React (Vite) dùng ES Module. Dự án NestJS 11 của bạn được biên dịch ra CommonJS, còn các package của NestJS 12 đã chuyển sang ESM. Bạn sẽ gặp cả hai.

1. So sánh

	CommonJS (CJS)	ES Module (ESM)
Cú pháp	`require()`, `module.exports`	`import`, `export`
Khi nào dùng	`.js` mặc định, `.cjs`	`.mjs`, hoặc `.js` khi `package.json` có `"type": "module"`
Tải module	Đồng bộ, lúc chạy đến dòng đó	Phân tích trước khi chạy, tải bất đồng bộ
Giá trị import	Bản sao tại thời điểm `require` (với primitive)	**Live binding:** tham chiếu đến biến gốc
`this` cấp file	`module.exports`	`undefined`
Strict mode	Không, trừ khi khai báo	Luôn bật
`await` cấp file	Không	Có
`__dirname`, `__filename`	Có	Không, dùng `import.meta.dirname`, `import.meta.filename`
Tree-shaking	Khó	Có, vì phân tích được trước khi chạy

> **So với PHP:** ESM giống `use` + autoload ở chỗ khai báo phụ thuộc ở đầu file. Khác biệt lớn là mỗi module JS có **scope riêng**: biến không `export` thì file khác không thấy. Trong PHP, `require` đưa code vào cùng scope.

2. Cú pháp ESM

MATH.MJS

```
export const PI = 3.14;
export function area(r) {
  return PI * r * r;
}
export default function add(a, b) {
  return a + b;
}
```

MAIN.MJS

```
import add, { PI, area as circleArea } from './math.mjs';
import * as math from './math.mjs';

console.log(add(1, 2), PI, circleArea(1));
console.log(Object.keys(math));
```

KẾT QUẢ KHI CHẠY

```
3 3.14 3.14
[ 'PI', 'area', 'default' ]
```

- Mỗi module có nhiều `export` có tên, nhưng chỉ một `export default`.
- Tên export phải đúng khi import, dùng `as` để đổi tên. Default thì đặt tên tuỳ ý.
- Trong Node, đường dẫn tương đối phải có **đuôi file** (`./math.mjs`). Vite và TypeScript có thể cho phép bỏ đuôi, vì đó là việc của công cụ, không phải chuẩn ESM.

3. Module chỉ chạy một lần

Lần `import` hoặc `require` đầu tiên chạy code của module rồi lưu kết quả vào cache. Các lần sau dùng lại kết quả đó. Vì vậy export một object từ module cho ta một **singleton**, mọi nơi dùng chung. `import` còn được "kéo lên đầu": module phụ thuộc chạy trước code của file hiện tại, dù `import` viết ở đâu.

4. Dùng lẫn CJS và ESM

- ESM import được CJS: `import pkg from './lib.cjs'`.
- CJS dùng ESM qua `await import()`. Node mới (từ 20.19 và 22.12) còn cho phép `require()` một ESM đồng bộ, nếu ESM đó không có `await` cấp file. Nhờ vậy dự án CJS vẫn dùng được package NestJS 12.

LIB.MJS

```
export const version = 'esm';
```

MAIN.CJS

```
const lib = require('./lib.mjs');
console.log(lib.version);

import('./lib.mjs').then((m) => console.log(m.version));
```

KẾT QUẢ KHI CHẠY

```
esm
esm
```

5. Dynamic import

`import()` trả về Promise, chỉ tải module khi cần. Đây là cơ sở của **code splitting** trong React: `React.lazy(() => import('./Page'))`.

Bài tập chương 8

Bài 8.1 **Tách module ESM** Viết code ★☆☆

Tạo file `price.mjs` export:

- `VAT = 0.1`
- hàm có tên `withVat(price)`
- `export default` hàm `format(n)` định dạng tiền kiểu `1.100 đ`

Biến `LOCALE` chỉ dùng nội bộ, không export. Trong `main.mjs`, in kết quả và chứng minh `LOCALE` không truy cập được từ ngoài.

KẾT QUẢ MONG ĐỢI

```
0.1
1.100 đ
[ 'VAT', 'default', 'withVat' ]
undefined undefined
```

Xem đáp án bài 8.1 →

Bài 8.2 CommonJS: giá trị bị sao chép

Dự đoán ★★☆

Dự đoán output. `count` là số (primitive), được export bằng `module.exports`.

COUNTER.CJS

```
let count = 0;
function inc() {
  count++;
}
function get() {
  return count;
}
module.exports = { count, inc, get };
```

MAIN.CJS

```
const { count, inc, get } = require('./counter.cjs');
const counter = require('./counter.cjs');

inc();
inc();
console.log(count);
console.log(counter.count);
console.log(get());
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 8.2 →

Bài 8.3 ESM: live binding

Dự đoán ★★☆

Cùng ý tưởng, viết bằng ESM. Dự đoán output, chú ý dòng gán.

COUNTER.MJS

```
export let count = 0;
export function inc() {
  count++;
}
```

MAIN.MJS

```
import { count, inc } from './counter.mjs';
import * as counter from './counter.mjs';

inc();
```

```
inc();
console.log(count);
console.log(counter.count);

try {
  counter.count = 100;
} catch (e) {
  console.log(e.name);
}
console.log(count);
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 8.3 →

Bài 8.4 Thứ tự chạy và cache module

Dự đoán ★★☆

Dự đoán output. `import` được viết sau `console.log`, và `config.mjs` được import ở hai nơi.

CONFIG.MJS

```
console.log('config chạy');
export const config = { env: 'dev' };
```

A.MJS

```
import { config } from './config.mjs';
console.log('a chạy');
config.env = 'prod';
```

B.MJS

```
import { config } from './config.mjs';
console.log('b chạy', config.env);
```

MAIN.MJS

```
console.log('main chạy');
import './a.mjs';
import './b.mjs';
```

Việc cần làm: ghi ra giấy kết quả bạn dự đoán, tự chạy để kiểm tra rồi mới xem đáp án.

Xem đáp án bài 8.4 →

Bài 8.5 `__dirname` trong ESM

Sửa lỗi ★☆☆

Code chuyển từ CommonJS sang ESM bị lỗi vì `__dirname` không tồn tại. Sửa dòng tạo `filePath`. Chương trình in tên file để kiểm tra.

CODE ĐỀ BÀI

```
import path from 'node:path';

let filePath;
try {
  filePath = path.join(__dirname, 'data.json');
} catch (e) {
  console.log(e.name);
```

```
}
console.log(filePath && path.basename(filePath));
```

KẾT QUẢ HIỆN TẠI (SAI)

```
ReferenceError
undefined
```

KẾT QUẢ MONG ĐỢI

```
data.json
```

Xem đáp án bài 8.5 →

Bài 8.6 **Tải driver theo cấu hình bằng dynamic import** Viết code ★★☆

Viết hàm `loadDriver(name)` dùng `import()` để tải đúng driver gửi mail theo tên. Tên không tồn tại thì ném lỗi rõ ràng. Hai file driver đã có sẵn. Đây là ý tưởng của `React.lazy` và lazy-loading module trong NestJS.

MAIL-LOG.MJS

```
console.log('Đã tải driver log');
export const send = (to) => `log: gửi cho ${to}`;
```

MAIL-SMTP.MJS

```
console.log('Đã tải driver smtp');
export const send = (to) => `smtp: gửi cho ${to}`;
```

KẾT QUẢ MONG ĐỢI

```
Bắt đầu
Đã tải driver log
log: gửi cho an@x.vn
true
Driver không hỗ trợ: sms
```

Xem đáp án bài 8.6 →

CHƯƠNG 9

Tổng hợp: tự viết bản thu nhỏ của React và NestJS

Cách tốt nhất để hết thấy "phép màu" là tự viết phiên bản đơn giản của nó. Các bài trong chương này dùng toàn bộ kiến thức từ chương 1–8, chỉ bằng JS thuần, chạy bằng Node. Mã thật của React và NestJS phức tạp hơn rất nhiều, các bài này chỉ mô phỏng **ý tưởng cốt lõi**.

Bài	Bản thu nhỏ của	Dùng kiến thức
9.1	`useState`, Rules of Hooks	Closure, mảng
9.2	`useEffect`, dependency array	Tham chiếu, `Object.is`, cleanup
9.3	`useReducer`, giỏ hàng	Bất biến, method mảng
9.4	Middleware của Express và NestJS	Hàm bậc cao, async
9.5	`EventEmitter` của Node	Closure, `Map`, `Set`
9.6	Dependency Injection của NestJS	Class, static, đệ quy
9.7	Service gọi API có phân trang	Async, `reduce`, xử lý lỗi

Làm theo thứ tự. Khi đọc React Part 03 hoặc NestJS Part 03, quay lại đối chiếu với code bạn đã tự viết.

Bài tập chương 9

Bài 9.1 Mini `useState` và Rules of Hooks

Viết code ★★★

Viết `createRuntime()` trả về `{ useState, render }`:

- `render(component)`: đặt lại con trỏ về 0, gọi `component()`, trả về kết quả.
- `useState(initial)`: lấy state theo **thứ tự gọi**, trả về `[value, setValue]`. `setValue` nhận giá trị hoặc hàm cập nhật.

Sau đó tạo component có hai state độc lập, và "render" nhiều lần để chứng minh state được giữ giữa các lần gọi.

KẾT QUẢ MONG ĐỢI

```
0 An
Biến của lần render cũ: 0
2 Bình
```

Xem đáp án bài 9.1 →

Bài 9.2 Mini `useEffect` và dependency array

Viết code ★★★

Mở rộng ý tưởng trên với `useEffect(fn, deps)`:

- Không có `deps`: chạy sau mọi lần render.
- `deps` không đổi (so sánh từng phần tử bằng `Object.is`): bỏ qua.
- Trước khi chạy lại, gọi hàm cleanup do lần trước trả về.

Component nhận props và có 3 effect để kiểm chứng. Effect chạy **sau** khi component trả về.

KẾT QUẢ MONG ĐỢI

```
render id=1
  effect không deps
  fetch job 1
  effect theo filter
render id=1
  effect không deps
render id=2
  effect không deps
  huỷ fetch job 1
  fetch job 2
  effect theo filter
```

Xem đáp án bài 9.2 →

Bài 9.3 Reducer giỏ hàng

Viết code ★★☆

Viết `cartReducer(state, action)` thuần, dùng được với `useReducer`:

- `add`: thêm sản phẩm, đã có thì cộng số lượng.
- `remove`: xoá theo `id`.
- `setQty`: đổi số lượng, `qty <= 0` thì xoá.
- Action không hỗ trợ thì ném lỗi.

Viết thêm `getTotal(state)`. Kiểm tra state cũ không bị đổi.

DỮ LIỆU / KHUNG CHO SẴN

```
const initial = { items: [] };
```

KẾT QUẢ MONG ĐỢI

```
200 400 300
1 3
true
Action không hỗ trợ: xyz
```

Xem đáp án bài 9.3 →

Bài 9.4 Middleware kiểu "củ hành"

Viết code ★★★

Viết `compose(middlewares)` trả về hàm `run(ctx)`, giống cơ chế middleware của Koa và ý tưởng interceptor của NestJS. Mỗi middleware có dạng `async (ctx, next) => {...}`. Code trước `await next()` chạy lúc đi vào, code sau chạy lúc đi ra. Không gọi `next` thì các lớp trong không chạy.

Dựng thử ba lớp: log thời gian, xác thực, và handler.

KẾT QUẢ MONG ĐỢI

```
--> vào /orders
handler chạy
<-- ra 200
--> vào /admin
<-- ra 401
```

Xem đáp án bài 9.4 →

Bài 9.5 Mini `EventEmitter`

Viết code ★★☆

Viết class `Emitter` có:

- `on(event, fn)`: đăng ký, trả về hàm huỷ.
- `once(event, fn)`: chỉ chạy một lần.
- `off(event, fn)`: huỷ đăng ký.
- `emit(event, ...args)`: gọi mọi listener theo thứ tự, trả về `true` nếu có listener.

Listener có thể tự huỷ trong lúc `emit` mà không làm hỏng vòng lặp.

KẾT QUẢ MONG ĐỢI

```
Gửi mail đơn 1
Chỉ một lần 1
true
Gửi mail đơn 2
false
false
```

Xem đáp án bài 9.5 →

Bài 9.6 Mini Dependency Injection container

Viết code ★★★

Viết class `Container` mô phỏng DI của NestJS. Mỗi class khai báo phụ thuộc trong `static inject = [...]`. NestJS làm việc này tự động bằng TypeScript decorator metadata (Part P.5), còn ở đây ta khai báo tay.

- `resolve(Token)`: tạo instance, đệ quy tạo các phụ thuộc trước.
- Instance là **singleton**.
- `override(Token, Other)`: thay implementation, dùng khi test.
- Phát hiện phụ thuộc vòng và ném lỗi rõ ràng.

KẾT QUẢ MONG ĐỢI

```
[ 'đơn thật' ]
true
true
[ 'đơn giả' ]
Phụ thuộc vòng tại A
```

Xem đáp án bài 9.6 →

Bài 9.7 Service tổng hợp từ API phân trang

Viết code ★★★

Viết `getTopCustomers(api, limit)`:

1. Gọi `api.getOrders(page)` lần lượt đến trang cuối. Dừng dựa vào `lastPage`.
2. Lỗi tạm thời ở một trang (`err.retryable === true`) thì thử lại tối đa 2 lần, lỗi khác thì ném ra ngay.
3. Chỉ tính đơn `paid`. Cộng tiền theo khách hàng, sắp giảm dần, lấy `limit` người đầu.

DỮ LIỆU / KHUNG CHO SẴN

```
const pages = {
  1: [{ customer: 'An', total: 100, status: 'paid' }, { customer: 'Bình', total: 300, status:
'new' }],
  2: [{ customer: 'Bình', total: 250, status: 'paid' }, { customer: 'An', total: 50, status:
'paid' }],
```

```
  3: [{ customer: 'Chi', total: 500, status: 'paid' }],
};

let failedOnce = false;
const api = {
  async getOrders(page) {
    if (page === 2 && !failedOnce) {
      failedOnce = true;
      const err = new Error('503');
      err.retryable = true;
      throw err;
    }
    return { data: pages[page], lastPage: 3 };
  },
};
```

KẾT QUẢ MONG ĐỢI

```
Thử lại lần 1: 503
[ { customer: 'Chi', total: 500 }, { customer: 'Bình', total: 250 } ]
```

Xem đáp án bài 9.7 →

PHẦN 2

ĐÁP ÁN

Chỉ mở sau khi đã tự làm hoặc đã kẹt quá 15 phút.

ĐÁP ÁN CHƯƠNG 1

Biến, scope và hoisting

Đáp án 1.1 Hoisting với `var`

KẾT QUẢ KHI CHẠY

```
undefined
An
undefined
```

Giải thích

`var name` được đăng ký trước khi chạy với giá trị `undefined`, phép gán vẫn nằm đúng ở dòng 2. Dòng đầu in `undefined`, dòng 3 in `An`.

Trong `show`, `var age` được hoist lên đầu **hàm**, chứ không lên đầu file. Khi `console.log(age)` chạy, `age` đã tồn tại nhưng chưa được gán, nên in `undefined`. Code tương đương như sau:

```
function show() {
  var age;           // khai báo được kéo lên
  console.log(age);  // undefined
  age = 20;          // phép gán giữ nguyên vị trí
}
```

← Về đề bài 1.1

Đáp án 1.2 Temporal Dead Zone

KẾT QUẢ KHI CHẠY

```
ngoài
ngoài
ReferenceError
```

Giải thích

`a` và `b` không khai báo `value` riêng nên tìm lên scope ngoài và in `ngoài`.

`c` khác ở một điểm: khối `try` có `const value` riêng. JS đã biết `value` thuộc khối này ngay từ đầu khối, nên không tìm ra ngoài nữa. Nhưng dòng `console.log` đứng trước dòng khai báo, đang nằm trong TDZ, nên ném `ReferenceError`.

Đây là bằng chứng `let` và `const` **vẫn được hoist**. Nếu không, `console.log(value)` đã in `ngoài` như hai hàm đầu.

← Về đề bài 1.2

Đáp án 1.3 Function declaration và function expression

KẾT QUẢ KHI CHẠY

```
declaration ok
var expression TypeError
```

```
const arrow ReferenceError
var expression sau gán ok
```

Giải thích

- `decl` là function declaration, được hoist cả thân hàm, gọi được ngay.
- `varExpr` được hoist với giá trị `undefined`. Gọi `undefined()` ném `TypeError` (biến có tồn tại, nhưng không phải hàm).
- `constArrow` còn trong TDZ nên ném `ReferenceError`.
- Sau dòng gán, `varExpr` đã là hàm nên chạy bình thường.

Phân biệt được `TypeError` và `ReferenceError` ở đây chính là hiểu hoisting: một bên biến đã có nhưng sai kiểu, một bên chưa được phép truy cập.

← Về đề bài 1.3

Đáp án 1.4 Block scope và shadowing

KẾT QUẢ KHI CHẠY

```
2 2
1 2
3 3
1 3
10
```

Giải thích

- Trong block, `let x` là biến mới, che `x` bên ngoài. `var y` không có block scope nên chính là `y` bên ngoài và ghi đè nó thành `2`.
- Ra khỏi block: `x` ngoài vẫn là `1`, còn `y` đã thành `2`.
- Trong `f`, `let x` là biến riêng của hàm. `y = 3` không khai báo, nên ghi vào `y` bên ngoài.
- Sau `f`: `x` là `1`, `y` là `3`.
- `let x` trong `for` có scope riêng của vòng lặp.

← Về đề bài 1.4

Đáp án 1.5 `var` trong vòng lặp

CODE ĐÁP ÁN

```
const handlers = [];
for (let i = 0; i < 3; i++) {
  handlers.push(() => i);
}
console.log(handlers.map((h) => h()));
```

KẾT QUẢ KHI CHẠY

```
[ 0, 1, 2 ]
```

Giải thích

Với `var`, ba hàm cùng giữ tham chiếu đến **một** biến `i`. Khi được gọi, vòng lặp đã xong và `i` là `3`. Với `let`, mỗi vòng có một `i` riêng nên mỗi hàm nhớ đúng giá trị của vòng đó.

Hàm vẫn giữ được biến sau khi vòng lặp kết thúc nhờ **closure**, sẽ học ở chương 4.

← Về đề bài 1.5

Đáp án 1.6 Biến toàn cục ngầm

KẾT QUẢ KHI CHẠY

```
100
100
ReferenceError
undefined
```

Giải thích

`total = 100` thiếu `let` nên ở sloppy mode, JS tạo thuộc tính `total` trên đối tượng toàn cục. Sau khi `sloppy` chạy, biến này dùng được ở khắp nơi. Đây là nguồn gốc của nhiều bug khó tìm: hai hàm vô tình dùng chung một biến.

Ở strict mode, cùng cách viết đó ném `ReferenceError`, nên `amount` không được tạo và `typeof amount` trả về `undefined`. Dùng `typeof` với biến chưa khai báo không gây lỗi.

← Về đề bài 1.6

Đáp án 1.7 Viết lại code `var` theo chuẩn hiện đại

CODE ĐÁP ÁN

```
const prices = [100, 200, 300];
let total = 0;
for (let i = 0; i < prices.length; i++) {
  const price = prices[i];
  total = total + price;
}
const label = 'Tổng: ' + total;
console.log(label);
```

KẾT QUẢ KHI CHẠY

```
Tổng: 600
```

Giải thích

- `prices`, `label` không gán lại nên dùng `const`.
- `total`, `i` thay đổi nên dùng `let`.
- `price` dùng `const` được dù nằm trong vòng lặp: mỗi vòng là một biến mới, không phải gán lại.

Lợi ích của `const`: người đọc biết chắc biến này không bị gán lại về sau, nên đọc code nhanh hơn.

← Về đề bài 1.7

ĐÁP ÁN CHƯƠNG 2

Kiểu dữ liệu, so sánh, tham chiếu và bất biến

Đáp án 2.1 `typeof`

KẾT QUẢ KHI CHẠY

```
number
string
object
undefined
object
object
function
number
function
```

Giải thích

Ba điểm hay sai:

- `typeof null` là `object`, một lỗi từ phiên bản đầu của JS, được giữ lại để không phá code cũ. Kiểm tra null bằng `x === null`.
- `typeof NaN` là `number`. NaN là giá trị đặc biệt thuộc kiểu số. Kiểm tra bằng `Number.isNaN(x)`.
- `class` là `function`. Class trong JS được xây trên function (chương 5).

← Về đề bài 2.1

Đáp án 2.2 `==` và `===`

KẾT QUẢ KHI CHẠY

```
true
false
true
true
true
false
false
false
false
false
```

Giải thích

- `==` ép chuỗi `'1'` và `''` thành số, và `false` thành `0`, nên ba phép so sánh tương ứng đều `true`.
- `null == undefined` là trường hợp đặc biệt được định nghĩa là `true`, nhưng `null == 0` là `false`: `null` chỉ bằng lỏng với `undefined`.
- `NaN` không bằng bất kỳ giá trị nào, kể cả chính nó.
- `[] == []` và `{} === {}` là hai object khác nhau nên đều `false`, dù nội dung giống nhau.

Cần so sánh nội dung hai object thì phải tự so từng trường, hoặc dùng `util.isDeepStrictEqual` trong Node.

← Về đề bài 2.2

Đáp án 2.3 Truthy và falsy

KẾT QUẢ KHI CHẠY

```
0 false
"" false
"0" true
"false" true
[] true
{} true
null false
undefined false
null false
-1 true
```

Giải thích

`'0'`, `'false'`, `[]`, `{}` và `-1` đều truthy. Trong PHP, `'0'` và `[]` là falsy, nên đây là chỗ dễ nhầm.

`JSON.stringify(NaN)` và `JSON.stringify(null)` đều cho chữ `null`, còn `JSON.stringify(undefined)` trả về `undefined`. Đó là lý do các dòng đó in như vậy.

← Về đề bài 2.3

Đáp án 2.4 `||` và `??` khi có giá trị 0

CODE ĐÁP ÁN

```js
function withDefaults(item) {
  return {
    name: item.name ?? 'Không tên',
    quantity: item.quantity ?? 1,
    discount: item.discount ?? 10,
  };
}

console.log(withDefaults({ name: 'A', quantity: 0, discount: 0 }));
console.log(withDefaults({}));
```

KẾT QUẢ KHI CHẠY

```
{ name: 'A', quantity: 0, discount: 0 }
{ name: 'Không tên', quantity: 1, discount: 10 }
```

Giải thích

`||` coi `0`, `''` và `false` là "không có", còn `??` chỉ coi `null` và `undefined` là "không có". Với dữ liệu số hoặc boolean, gần như luôn nên dùng `??`.

Nếu muốn chuỗi rỗng cũng bị thay bằng `'Không tên'`, thì dùng `||` riêng cho `name` lại hợp lý. Hãy chọn theo ý nghĩa nghiệp vụ.

← Về đề bài 2.4

Đáp án 2.5 Tham số là object

KẾT QUẢ KHI CHẠY

```
Bình
Bình
1
[ 1, 2 ] [ 1, 2 ] [ 1, 2, 3 ]
```

Giải thích

- `rename` sửa nội dung object được trỏ tới, nên `u.name` đổi thành `Bình`.
- `replace` gán tham số `user` sang object khác. Đây chỉ là biến cục bộ, `u` bên ngoài vẫn trỏ object cũ, nên vẫn là `Bình`.
- `bump` nhận bản sao của số, `count` không đổi.
- `alias` và `list` là cùng một mảng, nên `push` qua `alias` thì `list` cũng thấy. `copy` là mảng mới nên `push(3)` không ảnh hưởng.

JS truyền tham số **theo giá trị**, nhưng với object thì "giá trị" đó là tham chiếu. Điểm này giống object trong PHP và khác mảng trong PHP.

← Về đề bài 2.5

Đáp án 2.6 Bẫy sao chép nông và `JSON`

KẾT QUẢ KHI CHẠY

```
5
5
string
false
5
true
true
```

Giải thích

- `c1` sao chép nông, mảng `items` dùng chung, nên `source` cũng đổi thành `5`.
- `c2` qua JSON là mảng mới, `source` vẫn là `5`. Nhưng JSON làm **hỏng dữ liệu**: `Date` thành chuỗi, trường `undefined` biến mất.
- `c3` sao chép sâu đúng kiểu, `source` vẫn là `5`, `Date` còn nguyên, `note` vẫn tồn tại.

Dùng `structuredClone` khi cần sao chép sâu. Nó không sao chép được function: object có function sẽ ném lỗi `DataCloneError`.

← Về đề bài 2.6

Đáp án 2.7 Cập nhật state lồng nhau

CODE ĐÁP ÁN

```
const state = { job: { title: 'PHP', company: { name: 'Cũ', city: 'HN' } } };

function applyRename(s, name) {
  return {
    ...s,
    job: {
```

```
      ...s.job,
      company: { ...s.job.company, name },
    },
  };
}

const next = applyRename(state, 'Mới');
const same = next.job === state.job;
console.log(next.job.company.name, state.job.company.name, same);
```

KẾT QUẢ KHI CHẠY

```
Mới Cũ false
```

Giải thích

`{ ...s }` chỉ tạo mới tầng ngoài cùng, `next.job` vẫn là object cũ. Quy tắc là **tạo mới mọi tầng trên đường đi đến chỗ cần sửa** (`s`, `job`, `company`), các nhánh khác dùng lại tham chiếu cũ.

Dữ liệu lồng quá sâu thì nên làm phẳng cấu trúc state, hoặc dùng thư viện Immer.

← Về đề bài 2.7

Đáp án 2.8 Thêm, sửa, xoá mà không đổi mảng gốc

CODE ĐÁP ÁN

```
const jobs = [
  { id: 1, title: 'PHP', salary: 100 },
  { id: 2, title: 'React', salary: 300 },
  { id: 3, title: 'Nest', salary: 200 },
];

const addJob = (list, job) => [...list, job];
const removeJob = (list, id) => list.filter((j) => j.id !== id);
const updateSalary = (list, id, salary) =>
  list.map((j) => (j.id === id ? { ...j, salary } : j));
const sortBySalaryDesc = (list) => list.toSorted((a, b) => b.salary - a.salary);

const ids = (list) => list.map((j) => j.id).join(',');

console.log(ids(addJob(jobs, { id: 4, title: 'Vue', salary: 150 })));
console.log(ids(removeJob(jobs, 2)));
console.log(updateSalary(jobs, 3, 999)[2].salary);
console.log(ids(sortBySalaryDesc(jobs)));
console.log(ids(jobs), jobs[2].salary);
```

KẾT QUẢ KHI CHẠY

```
1,2,3,4
1,3
999
2,3,1
1,2,3 200
```

Giải thích

`{ ...j, salary }` là viết tắt của `{ ...j, salary: salary }`. Trường đứng sau ghi đè trường cùng tên đứng trước.

`toSorted` có từ Node 20. Môi trường cũ hơn thì viết `[...list].sort(fn)`. Cũng phải sao chép trước khi `sort`, vì `sort` sửa trực tiếp mảng.

← Về đề bài 2.8

Đáp án 2.9 Cộng tiền không sai số

CODE ĐÁP ÁN

```
const prices = [10.1, 20.2, 0.3];
let total = 0;
for (const p of prices) {
  total += Math.round(p * 1000);   // đổi sang đồng, số nguyên
}
console.log(total);
console.log(total === 30600);
console.log(`${total.toLocaleString('vi-VN')} đ`);
```

KẾT QUẢ KHI CHẠY

```
30600
true
30.600 đ
```

Giải thích

Số thực nhị phân không biểu diễn chính xác được `0.1`, sai số nhỏ tích luỹ khi cộng dồn. Cách chuẩn với tiền: **lưu và tính bằng số nguyên** ở đơn vị nhỏ nhất, chỉ định dạng khi hiển thị.

`Math.round` cần thiết vì bản thân `10.1 * 1000` có thể chưa ra đúng số nguyên. Cột tiền trong database cũng nên dùng `BIGINT` hoặc `DECIMAL`, không dùng `FLOAT`.

← Về đề bài 2.9

ĐÁP ÁN CHƯƠNG 3

Hàm, destructuring và method mảng

Đáp án 3.1 `sort` mặc định

KẾT QUẢ KHI CHẠY

```
[ 100, 20, 3 ]
[ 100, 20, 3 ]
[ 100, 20, 3 ] [ 3, 20, 100 ]
true [ 3, 2, 1 ]
```

Giải thích

- `sort()` mặc định so sánh theo chuỗi: `'100' < '20' < '3'`, nên kết quả là `[100, 20, 3]`. Nó cũng sửa trực tiếp `a`.
- `toSorted` trả mảng mới, `b` giữ nguyên.
- `reverse` sửa mảng gốc và trả về **chính mảng đó**, nên `r === c` là `true`.

← Về đề bài 3.1

Đáp án 3.2 `map` với `parseInt`

KẾT QUẢ KHI CHẠY

```
[ 1, NaN, NaN ]
[ 1, 2, 3 ]
[ 1, 2, 3 ]
```

Giải thích

`map` gọi callback với **3 tham số**: `(value, index, array)`. `parseInt` nhận 2 tham số: `(string, radix)`. Nên thực tế chạy như sau:

- `parseInt('1', 0)`: radix `0` nghĩa là tự nhận diện, kết quả `1`.
- `parseInt('2', 1)`: radix `1` không hợp lệ, kết quả `NaN`.
- `parseInt('3', 2)`: hệ nhị phân không có chữ số `3`, kết quả `NaN`.

`Number` chỉ dùng tham số đầu nên chạy đúng. Khi truyền hàm có sẵn làm callback, hãy kiểm tra hàm đó có tham số thứ hai không. An toàn nhất là viết arrow function rõ ràng.

← Về đề bài 3.2

Đáp án 3.3 Destructuring tham số

CODE ĐÁP ÁN

```
const a = { title: 'PHP Dev', salary: 15, company: { name: 'NKK', city: 'Hà Nội' } };
const b = { title: 'React Dev', company: { name: 'ABC' } };

function formatJob({ title, salary = 0, company: { name, city = 'Chưa rõ' } }) {
  return `${title} - ${salary} triệu - ${name} (${city})`;
}
```

```
console.log(formatJob(a));
console.log(formatJob(b));
```

KẾT QUẢ KHI CHẠY

```
PHP Dev - 15 triệu - NKK (Hà Nội)
React Dev - 0 triệu - ABC (Chưa rõ)
```

Giải thích

Giá trị mặc định chỉ áp dụng khi trường là `undefined`, không áp dụng khi là `null`. Nếu thiếu cả `company` thì code này ném `TypeError`, muốn an toàn phải viết `company: { name, city = 'Chưa rõ' } = {}`.

Đây đúng là cách viết props trong React: `function JobCard({ title, salary = 0 })`.

← Về đề bài 3.3

Đáp án 3.4 Lọc, lấy trường, sắp xếp

CODE ĐÁP ÁN

```
const jobs = [
  { title: 'Vue', salary: 90 },
  { title: 'React', salary: 300 },
  { title: 'Nest', salary: 200 },
  { title: 'Laravel', salary: 150 },
  { title: 'jQuery', salary: 50 },
];

const titles = jobs
  .filter((j) => j.salary > 100)
  .map((j) => j.title)
  .toSorted((a, b) => a.localeCompare(b));

console.log(titles);
console.log(jobs.map((j) => j.title));
```

KẾT QUẢ KHI CHẠY

```
[ 'Laravel', 'Nest', 'React' ]
[ 'Vue', 'React', 'Nest', 'Laravel', 'jQuery' ]
```

Giải thích

Nên lọc trước rồi mới biến đổi, để `map` không làm việc thừa. `localeCompare` so sánh theo thứ tự chữ cái thật. So sánh chuỗi bằng `<` theo mã ký tự thì chữ hoa đứng trước mọi chữ thường, và chữ có dấu sắp sai.

Ở đây `filter` đã tạo mảng mới nên dùng `sort` cũng không hại mảng gốc. `toSorted` chỉ giúp ý định rõ ràng hơn.

← Về đề bài 3.4

Đáp án 3.5 Tự viết `groupBy` bằng `reduce`

CODE ĐÁP ÁN

```
const orders = [
  { id: 1, status: 'paid', total: 100 },
  { id: 2, status: 'new', total: 50 },
```

```
  { id: 3, status: 'paid', total: 200 },
  { id: 4, status: 'cancel', total: 80 },
];

function groupBy(list, keyFn) {
  return list.reduce((acc, item) => {
    const key = keyFn(item);
    (acc[key] ??= []).push(item);
    return acc;
  }, {});
}

const groups = groupBy(orders, (o) => o.status);
console.log(Object.keys(groups));
console.log(groups.paid.map((o) => o.id));

const totals = Object.fromEntries(
  Object.entries(groups).map(([status, items]) => [
    status,
    items.reduce((s, o) => s + o.total, 0),
  ])
);
console.log(totals);
```

KẾT QUẢ KHI CHẠY

```
[ 'paid', 'new', 'cancel' ]
[ 1, 3 ]
{ paid: 300, new: 50, cancel: 80 }
```

Giải thích

`acc[key] ??= []` nghĩa là: nếu chưa có thì tạo mảng rỗng. Biểu thức này trả về mảng đó nên `push` được ngay.

Sửa `acc` ngay trong `reduce` là chấp nhận được vì `acc` do chính hàm này tạo ra. Viết `{ ...acc, [key]: [...] }` ở mỗi vòng cũng đúng, nhưng sao chép lại toàn bộ nên chậm với dữ liệu lớn.

← Về đề bài 3.5

Đáp án 3.6 Đếm và loại trùng

CODE ĐÁP ÁN

```
const posts = [
  { tags: ['js', 'react'] },
  { tags: ['php', 'js'] },
  { tags: ['js', 'nest', 'react'] },
];

const all = posts.flatMap((p) => p.tags);
console.log([...new Set(all)]);

const counts = all.reduce((acc, t) => {
  acc[t] = (acc[t] ?? 0) + 1;
  return acc;
}, {});
console.log(counts);
```

```
const [topTag] = Object.entries(counts).toSorted((a, b) => b[1] - a[1])[0];
console.log(topTag);
```

KẾT QUẢ KHI CHẠY

```
[ 'js', 'react', 'php', 'nest' ]
{ js: 3, react: 2, php: 1, nest: 1 }
js
```

Giải thích

- `flatMap` bằng `map` rồi `flat(1)`, rất hợp khi mỗi phần tử sinh ra nhiều giá trị.
- `Set` chỉ giữ giá trị duy nhất và giữ thứ tự thêm vào. So sánh theo `===` nên không dùng để loại trùng object theo nội dung được.

← Về đề bài 3.6

Đáp án 3.7 Lọc và đổi key của object

CODE ĐÁP ÁN

```
const body = { name: 'An', email: 'a@x.vn', isAdmin: true, password: '123' };

const pick = (obj, keys) =>
  Object.fromEntries(Object.entries(obj).filter(([k]) => keys.includes(k)));

const omit = (obj, keys) =>
  Object.fromEntries(Object.entries(obj).filter(([k]) => !keys.includes(k)));

console.log(pick(body, ['name', 'email']));
console.log(omit(body, ['password', 'isAdmin']));
console.log(body.isAdmin);
```

KẾT QUẢ KHI CHẠY

```
{ name: 'An', email: 'a@x.vn' }
{ name: 'An', email: 'a@x.vn' }
true
```

Giải thích

`[k]` destructuring chỉ lấy phần tử đầu của cặp `[key, value]`. Đừng dùng cách xoá key bằng `delete` trên object gốc, vì như vậy là sửa dữ liệu đầu vào.

Một cách khác để bỏ key: `const { password, ...safe } = body;`. Cách này rất hay gặp khi loại mật khẩu trước khi trả response.

← Về đề bài 3.7

Đáp án 3.8 `sort` làm hỏng dữ liệu gốc

CODE ĐÁP ÁN

```
const jobs = [
  { title: 'A', salary: 100 },
  { title: 'B', salary: 300 },
  { title: 'C', salary: 200 },
];
```

```
function topJobs(list, n) {
  return list.toSorted((a, b) => b.salary - a.salary).slice(0, n);
}

console.log(topJobs(jobs, 2).map((j) => j.title));
console.log(jobs.map((j) => j.title));
```

KẾT QUẢ KHI CHẠY

```
[ 'B', 'C' ]
[ 'A', 'B', 'C' ]
```

Giải thích

`sort` sửa trực tiếp mảng được truyền vào. Hàm nhận dữ liệu từ bên ngoài thì không nên có **tác dụng phụ** lên dữ liệu đó.

`slice` không có vấn đề này vì nó tạo mảng mới. Dễ nhầm giữa `slice` (không sửa gốc) và `splice` (có sửa gốc).

← Về đề bài 3.8

Đáp án 3.9 Phân trang và chia lô

CODE ĐÁP ÁN

```
const items = [1, 2, 3, 4, 5, 6, 7];

function paginate(list, page, perPage) {
  const start = (page - 1) * perPage;
  return {
    data: list.slice(start, start + perPage),
    page,
    perPage,
    total: list.length,
    lastPage: Math.ceil(list.length / perPage),
  };
}

function chunk(list, size) {
  const result = [];
  for (let i = 0; i < list.length; i += size) {
    result.push(list.slice(i, i + size));
  }
  return result;
}

console.log(paginate(items, 2, 3));
console.log(paginate(items, 4, 3).data);
console.log(chunk(items, 3));
```

KẾT QUẢ KHI CHẠY

```
{ data: [ 4, 5, 6 ], page: 2, perPage: 3, total: 7, lastPage: 3 }
[]
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7 ] ]
```

Giải thích

Trang vượt quá dữ liệu trả về mảng rỗng và không lỗi, vì `slice` tự xử lý chỉ số vượt biên.

`chunk` dùng `push` vẫn ổn vì `result` là mảng do hàm tự tạo, không phải dữ liệu đầu vào. Nguyên tắc bất biến chỉ áp dụng với dữ liệu nhận từ ngoài và state.

← Về đề bài 3.9

Đáp án 3.10 **Tự viết `map`, `filter`, `reduce`**

CODE ĐÁP ÁN

```
function myMap(arr, fn) {
  const out = [];
  for (let i = 0; i < arr.length; i++) out[i] = fn(arr[i], i, arr);
  return out;
}

function myFilter(arr, fn) {
  const out = [];
  for (let i = 0; i < arr.length; i++) {
    if (fn(arr[i], i, arr)) out[out.length] = arr[i];
  }
  return out;
}

function myReduce(arr, fn, ...init) {
  let i = 0;
  let acc;
  if (init.length > 0) {
    acc = init[0];
  } else {
    if (arr.length === 0) throw new TypeError('Mảng rỗng và không có giá trị đầu');
    acc = arr[0];
    i = 1;
  }
  for (; i < arr.length; i++) acc = fn(acc, arr[i], i, arr);
  return acc;
}

console.log(myMap([1, 2, 3], (x, i) => x * i));
console.log(myFilter([1, 2, 3, 4], (x) => x % 2 === 0));
console.log(myReduce([1, 2, 3], (s, x) => s + x));
console.log(myReduce([], (s, x) => s + x, 10));
try {
  myReduce([], (s, x) => s + x);
} catch (e) {
  console.log(e.name);
}
```

KẾT QUẢ KHI CHẠY

```
[ 0, 2, 6 ]
[ 2, 4 ]
6
10
TypeError
```

Giải thích

`...init` giúp phân biệt "không truyền giá trị đầu" với "truyền `undefined`", vì `init.length` khác nhau. Nếu viết `init === undefined` thì không phân biệt được hai trường hợp này.

Viết được `reduce` chứng tỏ bạn hiểu cơ chế của nó, không chỉ thuộc cú pháp.

← Về đề bài 3.10

ĐÁP ÁN CHƯƠNG 4

Closure

Đáp án 4.1 Hai counter độc lập

KẾT QUẢ KHI CHẠY

```
2 9
3
101 102
```

Giải thích

- `c1`, `c2` là hai lần gọi nên có hai biến `n` riêng. Ba hàm trong cùng một lần gọi dùng **chung** một `n`.
- `alias` trỏ cùng object với `c1`, nên `alias.inc()` đổi `n` của `c1`.
- `inc` lấy ra khỏi object vẫn chạy đúng vì nó dùng closure, không dùng `this`. Đây là ưu điểm so với viết bằng class (chương 5).

← Về đề bài 4.1

Đáp án 4.2 Closure trong vòng lặp `var`

KẾT QUẢ KHI CHẠY

```
[ 3, 3, 3 ]
[ 0, 1, 2 ]
[ 0, 1, 2 ]
```

Giải thích

- Cách 1: mọi hàm giữ cùng một biến `i`, cuối cùng là `3`.
- Cách 2: **IIFE** (hàm chạy ngay) tạo scope mới cho mỗi vòng, `k` là tham số nhận giá trị hiện tại của `j`. Bạn có thể gặp mẫu này trong code cũ.
- Cách 3: `let` tự tạo biến mới cho mỗi vòng, nên ngắn gọn nhất.

Nhớ ý chính: closure giữ **biến**, nên muốn giữ giá trị khác nhau thì phải có **biến khác nhau**.

← Về đề bài 4.2

Đáp án 4.3 `once`

CODE ĐÁP ÁN

```
function once(fn) {
  let called = false;
  let result;
  return (...args) => {
    if (!called) {
      called = true;
      result = fn(...args);
    }
    return result;
  };
}
```

```
const init = once((name) => {
  console.log('Khởi tạo', name);
  return 42;
});

console.log(init('a'));
console.log(init('b'));
```

KẾT QUẢ KHI CHẠY

```
Khởi tạo a
42
42
```

Giải thích

`called` và `result` được closure giữ qua các lần gọi. Dùng biến `called` riêng thay vì kiểm tra `result === undefined`, vì `fn` có thể trả về `undefined` thật.

Thực tế dùng để khởi tạo kết nối, hoặc chặn việc submit hai lần.

← Về đề bài 4.3

Đáp án 4.4 `memoize`

CODE ĐÁP ÁN

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

let calls = 0;
const slowAdd = (a, b) => {
  calls++;
  return a + b;
};
const fast = memoize(slowAdd);

console.log(fast(1, 2), fast(1, 2), fast(2, 3));
console.log('Số lần chạy thật:', calls);
```

KẾT QUẢ KHI CHẠY

```
3 3 5
Số lần chạy thật: 2
```

Giải thích

`cache.has` cần thiết vì kết quả có thể là `undefined` hoặc `0`. `Map` nhanh và cho phép khoá bất kỳ.

Hạn chế: cache lớn dần không giới hạn nên có thể gây tốn bộ nhớ trên server. `JSON.stringify` cũng không phân biệt được một số giá trị như `undefined` và `null` trong mảng. Thực tế cần thêm TTL hoặc giới hạn kích

thước (LRU).

← Về đề bài 4.4

Đáp án 4.5 `debounce`

CODE ĐÁP ÁN

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const search = debounce((q) => console.log('Gọi API với:', q), 50);

search('r');
search('re');
setTimeout(() => search('rea'), 20);
setTimeout(() => search('reac'), 200);
```

KẾT QUẢ KHI CHẠY

```
Gọi API với: rea
Gọi API với: reac
```

Giải thích

`timer` nằm trong closure nên các lần gọi dùng chung. Mỗi lần gọi huỷ timer cũ và đặt timer mới, nên chỉ lần cuối của mỗi đợt được chạy. Ở đây `rea` chạy sau 70 ms, còn `reac` thuộc một đợt riêng vì đến sau đó khá lâu.

Debounce chờ đến khi ngừng gọi, hợp với ô tìm kiếm. **Throttle** chạy tối đa một lần trong một khoảng thời gian, hợp với sự kiện cuộn trang.

Trong React, phải giữ hàm debounce qua các lần render (bằng `useRef` hoặc `useMemo`), và huỷ timer khi unmount.

← Về đề bài 4.5

Đáp án 4.6 **Stale closure khi cập nhật state**

CODE ĐÁP ÁN

```
let state = 0;
const setCount = (next) => {
  state = typeof next === 'function' ? next(state) : next;
};

function component() {
  const tick = () => setCount((c) => c + 1);
  return { tick };
}

const { tick } = component();
tick();
tick();
```

```
tick();
console.log(state);
```

KẾT QUẢ KHI CHẠY

```
3
```

Giải thích

Trong code cũ, `count` là hằng `0` của lần chạy đó, nên mỗi lần `tick` đều đặt state thành `0 + 1`. Khi truyền một hàm, `setCount` tự đưa vào giá trị mới nhất, nên `tick` không còn phụ thuộc vào biến cũ.

Trong React thật cũng vậy: `setCount(count + 1)` gọi 3 lần liên tiếp trong một lần render chỉ tăng 1, còn `setCount((c) => c + 1)` tăng 3.

← Về đề bài 4.6

Đáp án 4.7 Mini store kiểu Redux

CODE ĐÁP ÁN

```
function createStore(reducer, initial) {
  let state = initial;
  const listeners = new Set();
  return {
    getState: () => state,
    dispatch(action) {
      state = reducer(state, action);
      listeners.forEach((fn) => fn(state));
    },
    subscribe(fn) {
      listeners.add(fn);
      return () => listeners.delete(fn);
    },
  };
}

const reducer = (state, action) => {
  switch (action.type) {
    case 'add':
      return { ...state, count: state.count + action.by };
    case 'reset':
      return { ...state, count: 0 };
    default:
      return state;
  }
};

const store = createStore(reducer, { count: 0 });
const unsubscribe = store.subscribe((s) => console.log('Thay đổi:', s.count));

store.dispatch({ type: 'add', by: 2 });
store.dispatch({ type: 'add', by: 3 });
unsubscribe();
store.dispatch({ type: 'reset' });
console.log(store.getState());
```

KẾT QUẢ KHI CHẠY

```
Thay đổi: 2
Thay đổi: 5
{ count: 0 }
```

Giải thích

Sau `unsubscribe`, lần `reset` không in dòng "Thay đổi" nhưng state vẫn được cập nhật. Hàm huỷ đăng ký là closure nhớ `fn` và `listeners`.

Redux và Zustand về cốt lõi có cấu trúc giống hệt thế này. Mẫu "subscribe trả về hàm huỷ" gặp ở khắp nơi trong code React.

← Về đề bài 4.7

ĐÁP ÁN CHƯƠNG 5

`this`, class và prototype

Đáp án 5.1 Mất `this` khi lấy phương thức ra

KẾT QUẢ KHI CHẠY

```
Giỏ A
this là undefined
Giỏ B
this là undefined
```

Giải thích

- `cart.getName()`: object đứng trước dấu chấm là `cart`.
- `get()`: gọi trần nên `this` là `undefined`.
- `other.getName()`: cùng một hàm nhưng gọi qua `other`, nên `this` là `other`. Điều này chứng minh `this` phụ thuộc vào **cách gọi**, không phụ thuộc nơi định nghĩa hàm.
- `(0, cart.getName)()`: toán tử dấu phẩy trả về giá trị hàm, làm mất liên kết với object, nên giống gọi trần. Trong thực tế bạn thấy mẫu này trong code do bundler sinh ra.

← Về đề bài 5.1

Đáp án 5.2 Arrow function và `this`

KẾT QUẢ KHI CHẠY

```
this của file
obj
undefined
this của file
```

Giải thích

- `arrow` được viết trong dấu `{}` của object literal, nhưng **object literal không tạo scope**. Scope bao ngoài là cấp file. Trong file CommonJS, `this` ở cấp file là `module.exports`.
- `regular`: arrow `inner` lấy `this` của `regular`, tức `obj`. Đây là cách viết nên dùng.
- `broken`: `inner` là hàm thường và được gọi trần, nên `this` là `undefined`, dù nó nằm trong một phương thức.
- `arrow.call(...)`: `call` không đổi được `this` của arrow function, nên kết quả vẫn như cũ.

Quy tắc: không dùng arrow function làm phương thức của object. Dùng arrow function **bên trong** phương thức.

← Về đề bài 5.2

Đáp án 5.3 Mất `this` trong callback

CODE ĐÁP ÁN

```
class PriceService {
  constructor() {
    this.discount = 0.1;
  }
  apply(price) {
```

```
    return price * (1 - this.discount);
  }
}

const service = new PriceService();
try {
  console.log([100, 200].map((p) => service.apply(p)));
} catch (e) {
  console.log(e.name);
}
```

KẾT QUẢ KHI CHẠY

```
[ 90, 180 ]
```

Giải thích

`map` nhận hàm trần và gọi nó không kèm object. Code trong `class` luôn ở strict mode nên `this` là `undefined`, đọc `this.discount` thì ném `TypeError`.

Các cách sửa khác: `.map(service.apply.bind(service))`, hoặc `map` có tham số thứ hai đặt `this`: `.map(service.apply, service)`. Bọc arrow function là rõ ràng nhất.

Trong NestJS, mẫu `.then(this.handle)` hay `.pipe(map(this.transform))` cũng gặp đúng lỗi này.

← Về đề bài 5.3

Đáp án 5.4 Dùng `call`, `apply`, `bind`

CODE ĐÁP ÁN

```
function introduce(greeting, punctuation) {
  return `${greeting}, tôi là ${this.name}${punctuation}`;
}
const user = { name: 'An' };

console.log(introduce.call(user, 'Xin chào', '!'));
console.log(introduce.apply(user, ['Hello', '.']));

const hiAn = introduce.bind(user, 'Hi');
console.log(hiAn('?'));
console.log(hiAn.call({ name: 'Bình' }, '?'));
```

KẾT QUẢ KHI CHẠY

```
Xin chào, tôi là An!
Hello, tôi là An.
Hi, tôi là An?
Hi, tôi là An?
```

Giải thích

Dòng cuối cho thấy hàm đã `bind` **không đổi** `this` **được nữa**, kể cả dùng `call`.

Ngày nay `apply` ít dùng hơn vì đã có spread: `introduce.call(user, ...args)`.

← Về đề bài 5.4

Đáp án 5.5 Tự viết `bind`

CODE ĐÁP ÁN

```
function myBind(fn, ctx, ...preset) {
  return (...rest) => fn.call(ctx, ...preset, ...rest);
}

function total(a, b) {
  return `${this.label}: ${a + b}`;
}

const bound = myBind(total, { label: 'Tổng' }, 10);
console.log(bound(5));
console.log(bound(20));
```

KẾT QUẢ KHI CHẠY

```
Tổng: 15
Tổng: 30
```

Giải thích

Một bài kết hợp chương 4 và chương 5: closure giữ `fn`, `ctx`, `preset`, còn `call` đặt `this`. Viết hàm trả về bằng arrow function, vì ta không cần `this` riêng cho nó.

`bind` thật còn xử lý thêm trường hợp gọi bằng `new`. Không cần nhớ chi tiết đó ở mức này.

← Về đề bài 5.5

Đáp án 5.6 Class có private field và kế thừa

CODE ĐÁP ÁN

```
class Product {
  #price;

  constructor(name, price) {
    this.name = name;
    this.price = price;      // đi qua setter để được kiểm tra
  }

  get price() {
    return this.#price;
  }

  set price(value) {
    if (value < 0) throw new RangeError('Giá không được âm');
    this.#price = value;
  }

  describe() {
    return `${this.name}: ${this.price}`;
  }
}

class DiscountProduct extends Product {
  constructor(name, price, percent) {
    super(name, price);
```

```
    this.percent = percent;
  }

  describe() {
    return `${super.describe()} (giảm ${this.percent}%)`;
  }
}

const p = new DiscountProduct('Áo', 200, 10);
console.log(p.describe());

try {
  p.price = -1;
} catch (e) {
  console.log(e.name, e.message);
}
console.log(p.price, Object.keys(p));
```

KẾT QUẢ KHI CHẠY

```
Áo: 200 (giảm 10%)
RangeError Giá không được âm
200 [ 'name', 'percent' ]
```

Giải thích

- Constructor gán `this.price = price` nên đi qua setter, nhờ đó giá khởi tạo cũng được kiểm tra.
- `Object.keys` và `JSON.stringify` không thấy `#price`. Muốn trả ra JSON phải tự viết `toJSON()`.
- Class con không truy cập trực tiếp được `#price` của class cha, phải đi qua getter. Khác `protected` của PHP và TypeScript.

← Về đề bài 5.6

Đáp án 5.7 Prototype chain

KẾT QUẢ KHI CHẠY

```
A.hello B.hi
false
true true
own hello
A.hello
patched
patched
```

Giải thích

- `b.hello` không có trên `b`, không có trên `B.prototype`, tìm thấy trên `A.prototype`.
- Phương thức nằm trên prototype nên `Object.hasOwn(b, 'hi')` là `false`.
- `b.hello = ...` tạo thuộc tính **ngay trên** `b`, che phương thức trên prototype. Instance khác không bị ảnh hưởng.
- Sửa `A.prototype.hello` ảnh hưởng đến mọi instance đang tìm đến đó. Vì vậy không nên sửa prototype của đối tượng dùng chung.
- `delete b.hello` xoá thuộc tính riêng, lần đọc sau lại tìm lên prototype và gặp bản đã sửa.

← Về đề bài 5.7

Đáp án 5.8 Class field arrow function và `this`

KẾT QUẢ KHI CHẠY

```
undefined
Lưu
true
false
false true
```

Giải thích

- `handleA` là phương thức thường, gọi trần nên `this` là `undefined` và trả về `undefined`.
- `handleB` là class field, được tạo trong lúc khởi tạo mỗi instance. Arrow function lấy `this` từ đó, nên `this` luôn là `b1`.
- Đổi lại, mỗi instance có **một bản hàm riêng** (`handleB` khác nhau, tốn thêm bộ nhớ), còn `handleA` dùng chung trên prototype.

Vì vậy class field arrow function từng phổ biến trong React class component. Trong NestJS, service là singleton nên cách nào cũng được, nhưng phương thức thường cộng với bọc arrow khi truyền callback vẫn phổ biến nhất.

← Về đề bài 5.8

ĐÁP ÁN CHƯƠNG 6

Promise và async/await

Đáp án 6.1 Chuỗi `then`

KẾT QUẢ KHI CHẠY

```
4
undefined
p
cuối
```

Giải thích

- `then` đầu trả `4`, `then` thứ hai in `4`.
- `then` thứ hai không có `return` nên `then` kế tiếp nhận `undefined`. Đây là lỗi hay gặp: quên `return` làm đứt dữ liệu của chuỗi.
- Trả về `Promise.resolve('p')` thì chuỗi chờ Promise đó, nên `then` sau nhận `'p'` chứ không nhận một Promise.

← Về đề bài 6.1

Đáp án 6.2 Luồng lỗi với `catch`

KẾT QUẢ KHI CHẠY

```
catch A
2 phục hồi
catch B
catch C
finally
```

Giải thích

- Reject `A` bỏ qua `then` số 1, đến `catch` đầu.
- `catch` có `return` nên chuỗi trở lại trạng thái thành công, in `2 phục hồi`.
- Throw `B` đến `catch` thứ hai. `catch` này lại throw `C`, nên `then` số 3 bị bỏ qua và `catch` cuối nhận `C`.
- `finally` luôn chạy, không nhận tham số.

Muốn lỗi đi tiếp sau khi log, phải ném lại trong `catch`. Nếu không, phần sau tưởng rằng mọi thứ đã thành công.

← Về đề bài 6.2

Đáp án 6.3 `async` trả về gì

KẾT QUẢ KHI CHẠY

```
Promise { 1 }
true
object false
lỗi b
c = 2
```

Giải thích

- `a()` trả về Promise đã được resolve, `console.log` trong Node in `Promise { 1 }`.
- Mọi dòng đồng bộ, kể cả dòng `console.log` cuối file, đều in trước. `then` và `catch` chỉ chạy sau khi code đồng bộ kết thúc (chi tiết ở chương 7).
- `const r = c()` thiếu `await` nên `r` là Promise, không phải số `2`. Đây chính là lỗi quên `await`.

← Về đề bài 6.3

Đáp án 6.4 `forEach` không chờ `async`

CODE ĐÁP ÁN

```
const saved = [];
const save = (x) =>
  new Promise((r) => setTimeout(() => {
    saved.push(x);
    r();
  }, 10));

async function saveAll(items) {
  await Promise.all(items.map((x) => save(x)));
  console.log('Đã lưu xong', saved.length);
}

saveAll([1, 2, 3]).then(() => setTimeout(() => console.log('Thực tế', saved.length), 50));
```

KẾT QUẢ KHI CHẠY

```
Đã lưu xong 3
Thực tế 3
```

Giải thích

`forEach` gọi callback rồi bỏ qua giá trị trả về, nên các Promise không được ai chờ.

- Song song: `await Promise.all(items.map(...))`.
- Tuần tự, khi thứ tự quan trọng hoặc tránh quá tải: `for (const x of items) await save(x);`.

ESLint rule `no-floating-promises` trong dự án NestJS của bạn bắt được một số dạng lỗi này. Đừng tắt rule đó.

← Về đề bài 6.4

Đáp án 6.5 `return` so với `return await` trong `try`

CODE ĐÁP ÁN

```
const query = async () => {
  throw new Error('DB lỗi');
};

async function load() {
  try {
    return await query();
  } catch (e) {
    console.log('Bắt được trong load');
    return 'dự phòng';
  }
```

```
}

load()
  .then((v) => console.log('Kết quả:', v))
  .catch((e) => console.log('Lỗi lọt ra ngoài:', e.message));
```

KẾT QUẢ KHI CHẠY

```
Bắt được trong load
Kết quả: dự phòng
```

Giải thích

`return query()` trả Promise ra ngoài ngay, `try` kết thúc trước khi Promise reject, nên `catch` không kịp bắt. `return await` bắt hàm dừng lại **bên trong** `try` cho đến khi có kết quả.

Ngoài `try/catch`, `return promise` là bình thường, không cần thêm `await`.

← Về đề bài 6.5

Đáp án 6.6 Lấy kết quả khi có phần bị lỗi

CODE ĐÁP ÁN

```
const fetchUser = (id) =>
  id % 2 === 0
    ? Promise.resolve({ id, name: `User ${id}` })
    : Promise.reject(new Error(`Không tìm thấy ${id}`));
const ids = [1, 2, 3, 4];

async function main() {
  const results = await Promise.allSettled(ids.map(fetchUser));

  results.forEach((r, i) => {
    console.log(ids[i], r.status, r.status === 'fulfilled' ? r.value.name : r.reason.message);
  });

  const ok = results.filter((r) => r.status === 'fulfilled').map((r) => r.value);
  const failed = results.filter((r) => r.status === 'rejected').map((r) => r.reason.message);
  console.log(ok.length, failed);

  try {
    await Promise.all(ids.map(fetchUser));
  } catch (e) {
    console.log('Promise.all:', e.message);
  }
}

main();
```

KẾT QUẢ KHI CHẠY

```
1 rejected Không tìm thấy 1
2 fulfilled User 2
3 rejected Không tìm thấy 3
4 fulfilled User 4
2 [ 'Không tìm thấy 1', 'Không tìm thấy 3' ]
Promise.all: Không tìm thấy 1
```

Giải thích

`allSettled` luôn resolve, mỗi phần tử có `status` kèm `value` hoặc `reason`, đúng thứ tự mảng đầu vào.

`Promise.all` reject ngay với lỗi **đầu tiên**, kết quả thành công của các cái khác bị mất. Lưu ý các tác vụ khác vẫn chạy tiếp, `Promise.all` không huỷ được chúng.

← Về đề bài 6.6

Đáp án 6.7 Timeout bằng `Promise.race`

CODE ĐÁP ÁN

```
function withTimeout(promise, ms) {
  let timer;
  const timeout = new Promise((_, reject) => {
    timer = setTimeout(() => reject(new Error('Timeout')), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

const sleep = (ms, v) => new Promise((r) => setTimeout(() => r(v), ms));

async function main() {
  console.log(await withTimeout(sleep(10, 'nhanh'), 100));
  try {
    await withTimeout(sleep(200, 'chậm'), 20);
  } catch (e) {
    console.log(e.message);
  }
}

main();
```

KẾT QUẢ KHI CHẠY

```
nhanh
Timeout
```

Giải thích

`race` lấy kết quả của cái chốt trước. `finally` xoá timer dù kết quả thế nào.

Hạn chế: Promise thua cuộc **vẫn chạy tiếp**, `race` chỉ không còn quan tâm đến kết quả của nó. Muốn huỷ thật request `fetch` thì dùng `AbortController` hoặc `AbortSignal.timeout(ms)`.

← Về đề bài 6.7

Đáp án 6.8 `retry` có backoff

CODE ĐÁP ÁN

```
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

async function retry(fn, times = 3, delay = 10) {
  for (let attempt = 1; ; attempt++) {
    try {
      return await fn();
    } catch (e) {
      if (attempt >= times) throw e;
```

```
      console.log(`Lần ${attempt} lỗi, chờ ${delay} ms`);
      await sleep(delay);
      delay *= 2;
    }
  }
}

let count = 0;
const flaky = async () => {
  count++;
  if (count < 3) throw new Error(`lỗi ${count}`);
  return 'thành công';
};

async function main() {
  console.log(await retry(flaky));
  try {
    await retry(async () => { throw new Error('luôn lỗi'); }, 2);
  } catch (e) {
    console.log('Bỏ cuộc:', e.message);
  }
}

main();
```

KẾT QUẢ KHI CHẠY

```
Lần 1 lỗi, chờ 10 ms
Lần 2 lỗi, chờ 20 ms
thành công
Lần 1 lỗi, chờ 10 ms
Bỏ cuộc: luôn lỗi
```

Giải thích

`return await` trong `try` là bắt buộc, nếu không `catch` sẽ không bắt được lỗi (xem bài trước).

Backoff tránh dội thêm tải lên hệ thống đang lỗi. Thực tế chỉ retry lỗi tạm thời (mạng, 429, 503), không retry lỗi 400. Queue BullMQ trong NestJS có sẵn cơ chế này.

← Về đề bài 6.8

Đáp án 6.9 Giới hạn số việc chạy đồng thời

CODE ĐÁP ÁN

```
async function mapLimit(items, limit, fn) {
  const results = new Array(items.length);
  let next = 0;

  async function worker() {
    while (next < items.length) {
      const i = next++;
      results[i] = await fn(items[i], i);
    }
  }

  const workers = Array.from({ length: Math.min(limit, items.length) }, worker);
  await Promise.all(workers);
```

```
  return results;
}

let running = 0;
let peak = 0;
const job = async (x) => {
  running++;
  peak = Math.max(peak, running);
  await new Promise((r) => setTimeout(r, 10));
  running--;
  return x * 2;
};

mapLimit([1, 2, 3, 4, 5, 6, 7], 3, job).then((res) => {
  console.log(res);
  console.log('Đồng thời cao nhất:', peak);
});
```

KẾT QUẢ KHI CHẠY

```
[
   2,  4,  6, 8,
  10, 12, 14
]
Đồng thời cao nhất: 3
```

Giải thích

Tạo `limit` "công nhân", mỗi công nhân lấy việc kế tiếp cho đến khi hết. `next++` an toàn vì JS đơn luồng: đoạn giữa hai `await` không bị chen ngang (chương 7).

`results[i]` giữ đúng thứ tự dù việc xong không theo thứ tự. Thư viện thực tế: `p-limit`.

← Về đề bài 6.9

Đáp án 6.10 Bọc `fetch` đúng cách

CODE ĐÁP ÁN

```
const fakeFetch = async (url) =>
  url === '/jobs/1'
    ? { ok: true, status: 200, json: async () => ({ id: 1, title: 'PHP' }) }
    : { ok: false, status: 404, json: async () => ({ message: 'Not found' }) };

async function getJob(id) {
  const res = await fakeFetch(`/jobs/${id}`);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

async function main() {
  console.log(await getJob(1));
  try {
    console.log('Vâñ "thành công":', await getJob(2));
  } catch (e) {
    console.log('Lôĩ:', e.message);
  }
}
```

```
main();
```

KẾT QUẢ KHI CHẠY

```
{ id: 1, title: 'PHP' }
Lỗi: HTTP 404
```

Giải thích

`fetch` chỉ reject khi không có response (mất mạng, DNS, CORS). Có response thì dù 404 hay 500 cũng là thành công đối với Promise. `axios` thì ngược lại, nó tự reject với status ngoài khoảng 2xx.

Đây là lỗi rất phổ biến trong code React: giao diện hiển thị dữ liệu rỗng thay vì báo lỗi.

← Về đề bài 6.10

ĐÁP ÁN CHƯƠNG 7

Event loop

Đáp án 7.1 Thứ tự cơ bản

KẾT QUẢ KHI CHẠY

```
a
d
c
e
b
f
```

Giải thích

- Đồng bộ trước: `a`, `d`.
- Microtask theo thứ tự đăng ký: `c`, `e`.
- Macrotask theo thứ tự đăng ký: `b`, `f`.

Thứ tự viết trong code không quyết định thứ tự chạy, **loại hàng đợi** mới quyết định.

← Về đề bài 7.1

Đáp án 7.2 Lồng microtask và macrotask

KẾT QUẢ KHI CHẠY

```
sync
p1
p2
p trong p1
t1
p trong t1
t2
t trong p1
```

Giải thích

1. Đồng bộ: `sync`.
2. Microtask: `p1` chạy, đăng ký thêm một timer và một microtask. Tiếp theo là `p2`, rồi đến `p trong p1`. Microtask mới được chạy **ngay trong đợt này**, trước mọi macrotask.
3. Macrotask `t1` chạy, sinh ra microtask.
4. Trước macrotask kế tiếp phải xử lý hết microtask: `p trong t1` chạy trước `t2`.
5. `t2`, cuối cùng là `t trong p1` vì được đăng ký sau.

← Về đề bài 7.2

Đáp án 7.3 `async/await` xen kẽ

KẾT QUẢ KHI CHẠY

```
script bắt đầu
one bắt đầu
```

```
two
script kết thúc
one kết thúc
promise
timeout
```

Giải thích

- `one()` chạy đồng bộ đến `await`: in `one bắt đầu`, gọi `two()` và in `two` (cũng đồng bộ).
- Tại `await`, `one` dừng. Phần còn lại được xếp thành microtask **trước** `promise`, vì nó đăng ký trước.
- `script kết thúc` in tiếp, vì code đồng bộ phải xong trước.
- Microtask: `one kết thúc`, rồi `promise`. Cuối cùng là `timeout`.

← Về đề bài 7.3

Đáp án 7.4 Đan xen hai chuỗi Promise

KẾT QUẢ KHI CHẠY

```
A1
B1
C1
A2
B2
A3
```

Giải thích

Ban đầu hàng đợi gồm `A1`, `B1`, `C1`. Khi `A1` chạy xong, `A2` mới được xếp vào **cuối** hàng. Tương tự, `B1` xong thì `B2` được xếp vào.

Vì vậy hai chuỗi chạy **đan xen** từng bước, không phải chuỗi A chạy xong rồi mới đến chuỗi B. Đây là lý do hai luồng async đồng thời có thể xen vào nhau tại mỗi `await` (xem bài cuối chương).

← Về đề bài 7.4

Đáp án 7.5 `nextTick` và Promise trong CommonJS

KẾT QUẢ KHI CHẠY

```
sync
nextTick
promise
microtask
timeout
```

Giải thích

Trong CommonJS, sau khi code đồng bộ xong, Node xử lý hàng đợi `nextTick` trước, rồi đến Promise microtask (`promise`, `microtask` theo thứ tự đăng ký), cuối cùng là `timeout`.

Tài liệu Node khuyên ưu tiên `queueMicrotask` thay vì `process.nextTick`. Gọi `nextTick` đệ quy có thể làm các việc khác, kể cả I/O, không bao giờ được chạy.

← Về đề bài 7.5

Đáp án 7.6 Cùng code đó trong ES Module

KẾT QUẢ KHI CHẠY

```
sync
promise
microtask
nextTick
timeout
```

Giải thích

Thứ tự đã đổi: `promise`, `microtask` chạy **trước** `nextTick`.

ES Module được tải bất đồng bộ, nên toàn bộ code của module chạy bên trong một microtask. Khi code đồng bộ xong, Node vẫn đang ở đợt xử lý microtask, nên chạy nốt các microtask rồi mới đến `nextTick`.

Bạn không cần viết code dựa vào chi tiết này, chỉ cần biết tồn tại khác biệt. NestJS 12 chuyển các package sang ESM, nên đừng viết code phụ thuộc vào thứ tự `nextTick` và Promise.

← Về đề bài 7.6

Đáp án 7.7 `setImmediate` trong callback I/O

KẾT QUẢ KHI CHẠY

```
sync
trong readFile
nextTick
promise
immediate
timeout
```

Giải thích

- `sync` chạy trước, `readFile` trả kết quả sau.
- Trong callback: dòng đồng bộ, rồi `nextTick`, rồi `promise`.
- Đang ở pha `poll`, pha kế tiếp là `check`, nên `immediate` chạy trước. `timeout` phải chờ vòng lặp quay lại pha `timers`.

Trong callback I/O, `setImmediate` **luôn** chạy trước `setTimeout(fn, 0)`. Ngoài I/O thì không có gì đảm bảo.

← Về đề bài 7.7

Đáp án 7.8 Code đồng bộ chặn timer

KẾT QUẢ KHI CHẠY

```
vòng lặp xong
timer chạy, chờ ít nhất 50 ms: true
```

Giải thích

Timer hết hạn ở mốc 10 ms, nhưng luồng đang bận với `while`, nên callback phải chờ. Nó chỉ chạy sau khi stack rỗng, tức sau 50 ms. Đó là lý do delay chỉ là thời gian **tối thiểu**.

Trên server, trong 50 ms đó **không request nào** được xử lý. Nếu mỗi request chặn 50 ms thì server chỉ xử lý được khoảng 20 request/giây, bất kể máy mạnh đến đâu.

← Về đề bài 7.8

Đáp án 7.9 Microtask làm đói macrotask

KẾT QUẢ KHI CHẠY

```
sync
micro 1
micro 2
micro 3
timeout
```

Giải thích

Microtask sinh thêm microtask vẫn được chạy trong cùng đợt, nên `timeout` chỉ chạy khi hàng đợi microtask đã rỗng.

Nếu bỏ điều kiện `n < 3`, `timeout` sẽ **không bao giờ** chạy. Chương trình đơ mà không có lỗi. Cùng là vòng lặp vô hạn, nhưng dùng `setTimeout` đệ quy thì các việc khác vẫn chạy được, vì mỗi lần lặp là một macrotask.

← Về đề bài 7.9

Đáp án 7.10 Chia nhỏ việc nặng để không chặn

CODE ĐÁP ÁN

```
function sumBlocking(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}

function sumChunked(n, size = 100000) {
  return new Promise((resolve) => {
    let total = 0;
    let i = 1;
    function step() {
      const end = Math.min(i + size - 1, n);
      for (; i <= end; i++) total += i;
      if (i > n) resolve(total);
      else setImmediate(step);
    }
    step();
  });
}

async function main() {
  const N = 5000000;

  let beat = false;
  setTimeout(() => { beat = true; }, 0);
  const a = sumBlocking(N);
  console.log('Chặn:', a, '- heartbeat đã chạy:', beat);

  await new Promise((r) => setTimeout(r, 5));

  beat = false;
  setTimeout(() => { beat = true; }, 0);
  const b = await sumChunked(N);
```

```
  console.log('Chia nhỏ:', b, '- heartbeat đã chạy:', beat);
}

main();
```

KẾT QUẢ KHI CHẠY

```
Chặn: 12500002500000 - heartbeat đã chạy: false
Chia nhỏ: 12500002500000 - heartbeat đã chạy: true
```

Giải thích

Cùng một kết quả, cùng một lượng tính toán. Bản chia nhỏ thậm chí tốn tổng thời gian hơn một chút, nhưng nhường luồng cho việc khác giữa các đoạn.

Dùng `setImmediate`, không dùng `await Promise.resolve()`: Promise là microtask nên vẫn không cho timer hay I/O chạy. Việc rất nặng nên đẩy sang `worker_threads` hoặc queue thay vì chia nhỏ.

← Về đề bài 7.10

Đáp án 7.11 Đơn luồng vẫn có race condition

CODE ĐÁP ÁN

```
let balance = 0;
const delay = () => new Promise((r) => setTimeout(r, 10));

async function deposit(amount) {
  await delay();
  balance += amount;
}

Promise.all([deposit(100), deposit(50)]).then(() => console.log(balance));
```

KẾT QUẢ KHI CHẠY

```
150
```

Giải thích

Trong code cũ, cả hai hàm đọc `balance = 0`, rồi cùng chờ, rồi lần lượt ghi đè: `0 + 100`, sau đó `0 + 50`. Đoạn đọc và ghi bị `await` chia cắt, nên có khe hở.

Sau khi sửa, `balance += amount` nằm trọn giữa hai `await`, không ai chen vào được. Đoạn code đồng bộ luôn là **nguyên tử** trong JS.

Trong thực tế, trạng thái thường nằm trong database, nên dạng lỗi này vẫn xảy ra giữa `SELECT` và `UPDATE`. Khi đó phải dùng transaction, khoá dòng, hoặc câu lệnh atomic `UPDATE ... SET balance = balance + ?`.

← Về đề bài 7.11

ĐÁP ÁN CHƯƠNG 8

Module: CommonJS và ES Module

Đáp án 8.1 Tách module ESM

PRICE.MJS

```
const LOCALE = 'vi-VN';

export const VAT = 0.1;

export function withVat(price) {
  return Math.round(price * (1 + VAT));
}

export default function format(n) {
  return `${n.toLocaleString(LOCALE)} đ`;
}
```

CODE ĐÁP ÁN (MAIN.MJS)

```
import format, { VAT, withVat } from './price.mjs';
import * as all from './price.mjs';

console.log(VAT);
console.log(format(withVat(1000)));
console.log(Object.keys(all));
console.log(typeof LOCALE, all.LOCALE);
```

KẾT QUẢ KHI CHẠY

```
0.1
1.100 đ
[ 'VAT', 'default', 'withVat' ]
undefined undefined
```

Giải thích

Biến không `export` là riêng của module, kể cả `import *` cũng không thấy. Đây là cách đóng gói tự nhiên, gần giống closure.

`Object.keys` có chứa `default`, vì `export default` thực chất là một export tên `default`.

← Về đề bài 8.1

Đáp án 8.2 CommonJS: giá trị bị sao chép

KẾT QUẢ KHI CHẠY

```
0
0
2
```

Giải thích

`module.exports = { count, ... }` tạo object với **giá trị `count` lúc đó**, là `0`. Biến `count` bên trong đổi về sau, nhưng thuộc tính của object không đổi theo. Cả `count` và `counter.count` vẫn là `0`.

`get()` là closure đọc biến thật nên ra `2`. Nếu thứ được export là object thì bên ngoài vẫn thấy nội dung thay đổi, vì lúc đó được sao chép là tham chiếu.

← Về đề bài 8.2

Đáp án 8.3 ESM: live binding

KẾT QUẢ KHI CHẠY

```
2
2
TypeError
2
```

Giải thích

ESM import là **live binding**, tức một liên kết đến biến gốc, không phải bản sao. `inc` đổi biến gốc nên `count` đọc ra `2`.

Bên ngoài chỉ được đọc. Gán vào namespace `counter.count` ném `TypeError`. Gán trực tiếp `count = 100` cũng ném `TypeError`, vì biến được import là chỉ đọc đối với file import nó. Chỉ module sở hữu biến mới được đổi nó, như `inc` ở đây.

← Về đề bài 8.3

Đáp án 8.4 Thứ tự chạy và cache module

KẾT QUẢ KHI CHẠY

```
config chạy
a chạy
b chạy prod
main chạy
```

Giải thích

- `import` được kéo lên đầu, nên các phụ thuộc chạy trước mọi dòng của `main`, dù `console.log` viết trước.
- `a` phụ thuộc `config`, nên `config` chạy trước `a`.
- `b` cũng import `config`, nhưng `config` đã có trong cache nên **không chạy lại**.
- `b` và `a` dùng chung một object, nên `b` thấy `env` đã bị đổi thành `prod`.

Một module export object chính là singleton. Lợi ích là chia sẻ dễ dàng, nhưng trạng thái dùng chung có thể bị sửa từ bất kỳ đâu. Trên server NestJS, trạng thái module dùng chung giữa **mọi request**, nên đừng lưu dữ liệu của từng user vào đó.

← Về đề bài 8.4

Đáp án 8.5 `__dirname` trong ESM

CODE ĐÁP ÁN

```
import path from 'node:path';
```

```
let filePath;
try {
  filePath = path.join(import.meta.dirname, 'data.json');
} catch (e) {
  console.log(e.name);
}
console.log(filePath && path.basename(filePath));
```

KẾT QUẢ KHI CHẠY

```
data.json
```

Giải thích

ESM không có `require`, `module`, `__dirname`, `__filename`. `import.meta.dirname` có từ Node 20.11. Trên Node cũ hơn phải dùng `fileURLToPath(import.meta.url)`.

Bạn sẽ gặp lỗi này khi chuyển dự án sang ESM. Chuyển đổi nên thay cả cách viết đường dẫn.

← Về đề bài 8.5

Đáp án 8.6 Tải driver theo cấu hình bằng dynamic import

CODE ĐÁP ÁN (MAIN.MJS)

```
const DRIVERS = {
  log: () => import('./mail-log.mjs'),
  smtp: () => import('./mail-smtp.mjs'),
};

async function loadDriver(name) {
  const load = DRIVERS[name];
  if (!load) throw new Error(`Driver không hỗ trợ: ${name}`);
  return load();
}

console.log('Bắt đầu');
const driver = await loadDriver('log');
console.log(driver.send('an@x.vn'));

const again = await loadDriver('log');
console.log(again === driver);

try {
  await loadDriver('sms');
} catch (e) {
  console.log(e.message);
}
```

KẾT QUẢ KHI CHẠY

```
Bắt đầu
Đã tải driver log
log: gửi cho an@x.vn
true
Driver không hỗ trợ: sms
```

Giải thích

- Chỉ driver được dùng mới được tải: dòng "Đã tải driver smtp" không xuất hiện.
- Tải lại cùng module trả về cùng một object, không chạy lại.
- `await` cấp file chỉ dùng được trong ESM.

Dùng bảng `DRIVERS` thay vì ghép chuỗi `import('./mail-' + name)` để không nạp nhầm file theo dữ liệu người dùng nhập, và để bundler như Vite biết trước cần đóng gói những file nào.

← Về đề bài 8.6

ĐÁP ÁN CHƯƠNG 9

Tổng hợp: tự viết bản thu nhỏ của React và NestJS

Đáp án 9.1 Mini `useState` và Rules of Hooks

CODE ĐÁP ÁN

```
function createRuntime() {
  const hooks = [];
  let cursor = 0;

  function useState(initial) {
    const i = cursor++;
    if (!(i in hooks)) hooks[i] = initial;
    const setValue = (next) => {
      hooks[i] = typeof next === 'function' ? next(hooks[i]) : next;
    };
    return [hooks[i], setValue];
  }

  function render(component) {
    cursor = 0;
    return component();
  }

  return { useState, render };
}

const { useState, render } = createRuntime();

function Counter() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('An');
  return { count, name, setCount, setName };
}

let ui = render(Counter);
console.log(ui.count, ui.name);

ui.setCount((c) => c + 1);
ui.setCount((c) => c + 1);
ui.setName('Bình');
console.log('Biến của lần render cũ:', ui.count);

ui = render(Counter);
console.log(ui.count, ui.name);
```

KẾT QUẢ KHI CHẠY

```
0 An
Biến của lần render cũ: 0
2 Bình
```

Giải thích

Ba điều về React được làm rõ:

- **State không đổi trong một lần render.** `ui.count` vẫn là `0` sau khi set. Giá trị mới chỉ xuất hiện ở lần render sau. Đó là lý do `console.log(count)` ngay sau `setCount` in giá trị cũ.
- **Hook xác định bằng thứ tự gọi, không bằng tên.** `useState` không biết tên biến của bạn, chỉ biết vị trí `cursor`. Đặt hook trong `if` làm lệch vị trí, hook sau sẽ đọc nhầm state. Đó chính là lý do của **Rules of Hooks**.
- **State nằm ngoài component**, trong closure của runtime. Vì vậy hàm component chạy lại mà state không mất.

← Về đề bài 9.1

Đáp án 9.2 Mini `useEffect` và dependency array

CODE ĐÁP ÁN

```
function createRuntime() {
  const hooks = [];
  let cursor = 0;
  let pending = [];

  function useEffect(fn, deps) {
    const i = cursor++;
    const prev = hooks[i];
    const changed =
      !prev || !deps || deps.some((d, k) => !Object.is(d, prev.deps[k]));
    if (changed) {
      pending.push(() => {
        prev?.cleanup?.();
        hooks[i] = { deps, cleanup: fn() };
      });
    }
  }

  function render(component, props) {
    cursor = 0;
    pending = [];
    const output = component(props);
    pending.forEach((run) => run());
    return output;
  }

  return { useEffect, render };
}

const { useEffect, render } = createRuntime();

function Job({ id, filter }) {
  console.log(`render id=${id}`);
  useEffect(() => console.log('  effect không deps'));
  useEffect(() => {
    console.log(`  fetch job ${id}`);
    return () => console.log(`  huỷ fetch job ${id}`);
  }, [id]);
  useEffect(() => console.log('  effect theo filter'), [filter]);
}

const filter = { q: 'php' };
render(Job, { id: 1, filter });
```

```
render(Job, { id: 1, filter });
render(Job, { id: 2, filter: { q: 'php' } });
```

KẾT QUẢ KHI CHẠY

```
render id=1
  effect không deps
  fetch job 1
  effect theo filter
render id=1
  effect không deps
render id=2
  effect không deps
  huỷ fetch job 1
  fetch job 2
  effect theo filter
```

Giải thích

Đọc output từng lần render:

- **Lần 1:** mọi effect chạy.
- **Lần 2:** `id` và `filter` giữ nguyên tham chiếu, chỉ effect không có `deps` chạy.
- **Lần 3:** `id` đổi nên cleanup của lần trước chạy, rồi fetch lại. `filter` có **nội dung giống hệt** nhưng là **object mới**, nên `Object.is` trả về `false` và effect chạy lại.

Dòng cuối chính là bug thường gặp: tạo object hay mảng ngay trong lúc render rồi đưa vào dependency làm effect chạy lại mỗi lần. Cách sửa: dùng giá trị primitive (`filter.q`) làm dependency, hoặc `useMemo` .

Bản này đã đơn giản hoá. React thật chạy **toàn bộ cleanup trước**, rồi mới chạy các effect, và effect chạy sau khi giao diện đã cập nhật.

← Về đề bài 9.2

Đáp án 9.3 Reducer giỏ hàng

CODE ĐÁP ÁN

```
const initial = { items: [] };

function cartReducer(state, action) {
  switch (action.type) {
    case 'add': {
      const found = state.items.some((i) => i.id === action.item.id);
      const items = found
        ? state.items.map((i) =>
            i.id === action.item.id ? { ...i, qty: i.qty + action.item.qty } : i
          )
        : [...state.items, action.item];
      return { ...state, items };
    }
    case 'remove':
      return { ...state, items: state.items.filter((i) => i.id !== action.id) };
    case 'setQty':
      return {
        ...state,
        items:
          action.qty <= 0
            ? state.items.filter((i) => i.id !== action.id)
```

```
            : state.items.map((i) => (i.id === action.id ? { ...i, qty: action.qty } : i)),
      };
    default:
      throw new Error(`Action không hỗ trợ: ${action.type}`);
  }
}

const getTotal = (state) => state.items.reduce((s, i) => s + i.price * i.qty, 0);

const s1 = cartReducer(initial, { type: 'add', item: { id: 1, price: 100, qty: 1 } });
const s2 = cartReducer(s1, { type: 'add', item: { id: 2, price: 50, qty: 2 } });
const s3 = cartReducer(s2, { type: 'add', item: { id: 1, price: 100, qty: 2 } });
const s4 = cartReducer(s3, { type: 'setQty', id: 2, qty: 0 });

console.log(getTotal(s2), getTotal(s3), getTotal(s4));
console.log(s1.items[0].qty, s3.items[0].qty);
console.log(s3.items[1] === s2.items[1]);

try {
  cartReducer(s4, { type: 'xyz' });
} catch (e) {
  console.log(e.message);
}
```

KẾT QUẢ KHI CHẠY

```
200 400 300
1 3
true
Action không hỗ trợ: xyz
```

Giải thích

Reducer **thuần**: cùng đầu vào cho cùng đầu ra, không gọi API, không sửa đầu vào. Nhờ đó dễ test như đoạn trên, không cần giao diện.

Không lưu `total` trong state, vì giá trị này tính được từ `items`. Lưu thêm dễ gây lệch dữ liệu. Sản phẩm không sửa giữ nguyên tham chiếu (dòng in `true`), nhờ vậy `React.memo` bỏ qua được các dòng không đổi.

← Về đề bài 9.3

Đáp án 9.4 Middleware kiểu "củ hành"

CODE ĐÁP ÁN

```
function compose(middlewares) {
  return function run(ctx) {
    function dispatch(i) {
      const mw = middlewares[i];
      if (!mw) return Promise.resolve();
      return Promise.resolve(mw(ctx, () => dispatch(i + 1)));
    }
    return dispatch(0);
  };
}

const logger = async (ctx, next) => {
  console.log('--> vào', ctx.path);
  await next();
```

```
  console.log('<-- ra', ctx.status);
};

const auth = async (ctx, next) => {
  if (!ctx.token) {
    ctx.status = 401;
    return;                 // không gọi next: chặn lại
  }
  await next();
};

const handler = async (ctx) => {
  console.log('handler chạy');
  ctx.status = 200;
};

const app = compose([logger, auth, handler]);

async function main() {
  await app({ path: '/orders', token: 'abc' });
  await app({ path: '/admin' });
}

main();
```

KẾT QUẢ KHI CHẠY

```
--> vào /orders
handler chạy
<-- ra 200
--> vào /admin
<-- ra 401
```

Giải thích

Request đi vào qua từng lớp, rồi đi ra theo chiều ngược lại, như các lớp của củ hành. `next` là closure nhớ `i`, gọi nó là chuyển sang lớp kế tiếp.

So sánh với NestJS: **Guard** giống `auth` (quyết định có cho đi tiếp không), **Interceptor** giống `logger` (bọc quanh handler, can thiệp cả lúc vào và ra). Middleware của Laravel cũng có cấu trúc tương tự với `$next($request)`.

← Về đề bài 9.4

Đáp án 9.5 Mini `EventEmitter`

CODE ĐÁP ÁN

```
class Emitter {
  #listeners = new Map();

  on(event, fn) {
    if (!this.#listeners.has(event)) this.#listeners.set(event, new Set());
    this.#listeners.get(event).add(fn);
    return () => this.off(event, fn);
  }

  once(event, fn) {
    const off = this.on(event, (...args) => {
```

```
      off();
      fn(...args);
    });
    return off;
  }

  off(event, fn) {
    this.#listeners.get(event)?.delete(fn);
  }

  emit(event, ...args) {
    const set = this.#listeners.get(event);
    if (!set || set.size === 0) return false;
    [...set].forEach((fn) => fn(...args));
    return true;
  }
}

const bus = new Emitter();
const stop = bus.on('order', (id) => console.log('Gửi mail đơn', id));
bus.once('order', (id) => console.log('Chỉ một lần', id));

console.log(bus.emit('order', 1));
bus.emit('order', 2);
stop();
console.log(bus.emit('order', 3));
console.log(bus.emit('khong-co'));
```

KẾT QUẢ KHI CHẠY

```
Gửi mail đơn 1
Chỉ một lần 1
true
Gửi mail đơn 2
false
false
```

Giải thích

`[...set]` chụp lại danh sách trước khi lặp, nên listener tự huỷ không ảnh hưởng đến vòng lặp đang chạy.

`emit` chạy **đồng bộ**, cũng như `EventEmitter` của Node: listener chạy xong thì dòng sau `emit` mới chạy. Listener chậm hoặc ném lỗi sẽ ảnh hưởng đến người phát sự kiện. Điều này liên quan đến NestJS Part 19.

← Về đề bài 9.5

Đáp án 9.6 Mini Dependency Injection container

CODE ĐÁP ÁN

```
class Container {
  #instances = new Map();
  #overrides = new Map();
  #resolving = new Set();

  override(token, impl) {
    this.#overrides.set(token, impl);
  }
```

```
  resolve(token) {
    if (this.#instances.has(token)) return this.#instances.get(token);
    if (this.#resolving.has(token)) {
      throw new Error(`Phụ thuộc vòng tại ${token.name}`);
    }

    this.#resolving.add(token);
    const Impl = this.#overrides.get(token) ?? token;
    const deps = (Impl.inject ?? []).map((d) => this.resolve(d));
    const instance = new Impl(...deps);
    this.#resolving.delete(token);

    this.#instances.set(token, instance);
    return instance;
  }
}

class Database {
  find() { return ['đơn thật']; }
}
class OrderRepository {
  static inject = [Database];
  constructor(db) { this.db = db; }
  all() { return this.db.find(); }
}
class OrderService {
  static inject = [OrderRepository];
  constructor(repo) { this.repo = repo; }
  list() { return this.repo.all(); }
}

const c1 = new Container();
const service = c1.resolve(OrderService);
console.log(service.list());
console.log(c1.resolve(OrderService) === service);
console.log(c1.resolve(Database) === service.repo.db);

class FakeDatabase {
  find() { return ['đơn giả']; }
}
const c2 = new Container();
c2.override(Database, FakeDatabase);
console.log(c2.resolve(OrderService).list());

class A { static inject = []; }
class B { static inject = [A]; }
A.inject = [B];
try {
  new Container().resolve(A);
} catch (e) {
  console.log(e.message);
}
```

KẾT QUẢ KHI CHẠY

```
[ 'đơn thật' ]
true
true
```

```
[ 'đơn giả' ]
Phụ thuộc vòng tại A
```

Giải thích

Đây là ý tưởng của `@Injectable()` và `Test.createTestingModule().overrideProvider()`:

- `OrderService` không tự `new` phụ thuộc nên đổi được database giả khi test, không phải sửa service.
- Class trong JS là giá trị thật, nên dùng được làm **khoá của** `Map`. Interface của TypeScript biến mất sau khi biên dịch nên không làm token được. Đó là lý do NestJS cần abstract class hoặc token chuỗi.
- `#resolving` phát hiện vòng lặp, giống lỗi NestJS báo khi gặp circular dependency.

← Về đề bài 9.6

Đáp án 9.7 Service tổng hợp từ API phân trang

CODE ĐÁP ÁN

```
const pages = {
  1: [{ customer: 'An', total: 100, status: 'paid' }, { customer: 'Bình', total: 300, status:
'new' }],
  2: [{ customer: 'Bình', total: 250, status: 'paid' }, { customer: 'An', total: 50, status:
'paid' }],
  3: [{ customer: 'Chi', total: 500, status: 'paid' }],
};

let failedOnce = false;
const api = {
  async getOrders(page) {
    if (page === 2 && !failedOnce) {
      failedOnce = true;
      const err = new Error('503');
      err.retryable = true;
      throw err;
    }
    return { data: pages[page], lastPage: 3 };
  },
};

async function withRetry(fn, retries = 2) {
  for (let attempt = 0; ; attempt++) {
    try {
      return await fn();
    } catch (e) {
      if (!e.retryable || attempt >= retries) throw e;
      console.log(`Thử lại lần ${attempt + 1}: ${e.message}`);
    }
  }
}

async function fetchAll(api) {
  const all = [];
  let page = 1;
  let lastPage = 1;
  do {
    const res = await withRetry(() => api.getOrders(page));
    all.push(...res.data);
    lastPage = res.lastPage;
    page++;
```

```
  } while (page <= lastPage);
  return all;
}

async function getTopCustomers(api, limit) {
  const orders = await fetchAll(api);
  const totals = orders
    .filter((o) => o.status === 'paid')
    .reduce((acc, o) => {
      acc[o.customer] = (acc[o.customer] ?? 0) + o.total;
      return acc;
    }, {});

  return Object.entries(totals)
    .map(([customer, total]) => ({ customer, total }))
    .toSorted((a, b) => b.total - a.total)
    .slice(0, limit);
}

getTopCustomers(api, 2)
  .then(console.log)
  .catch((e) => console.log('Lỗi:', e.message));
```

KẾT QUẢ KHI CHẠY

```
Thử lại lần 1: 503
[ { customer: 'Chi', total: 500 }, { customer: 'Bình', total: 250 } ]
```

Giải thích

Tách thành 3 hàm nhỏ: `withRetry` xử lý lỗi, `fetchAll` xử lý phân trang, `getTopCustomers` xử lý nghiệp vụ. Mỗi hàm test riêng được. Đây là cách tổ chức của một service trong NestJS. Tham số `api` được truyền vào nên có thể thay bằng bản giả, đúng như bài DI vừa rồi.

Các trang được gọi **tuần tự** vì cần biết `lastPage` từ response. Biết trước số trang thì có thể chạy song song các trang sau kèm giới hạn đồng thời (bài `mapLimit` ở chương 6).

← Về đề bài 9.7

PHỤ LỤC

Bảng tra cứu và lỗi hay gặp

JavaScript và PHP

Việc	JavaScript	PHP
Khai báo biến	`const x = 1;` `let y = 2;`	`$x = 1;`
Nối chuỗi, nội suy	`` `Chào ${name}` ``	`"Chào $name"`, `'a' . 'b'`
So sánh chặt	`===`	`===`
Mảng danh sách và key-value	`Array` riêng, `Object` / `Map` riêng	`array` dùng cho cả hai
Gán mảng	**Cùng tham chiếu**	Sao chép
Falsy	`[]`, `'0'` là **truthy**	`[]`, `'0'` là falsy
Độ dài	`arr.length`, `str.length`	`count($a)`, `strlen($s)`
Biến đổi mảng	`arr.map(fn)`	`array_map($fn, $arr)`
Lọc mảng	`arr.filter(fn)`, đánh lại chỉ số	`array_filter($arr, $fn)`, giữ key
Gộp	`arr.reduce(fn, init)`	`array_reduce($arr, $fn, $init)`
Có chứa	`arr.includes(x)`	`in_array($x, $arr, true)`
Gộp mảng	`[...a, ...b]`, `{ ...a, ...b }`	`array_merge($a, $b)`, `[...$a, ...$b]`
Null an toàn	`a?.b ?? 'x'`	`$a?->b ?? 'x'`
Hàm ẩn danh	`(x) => x * 2`, tự thấy biến ngoài	`fn ($x) => $x * 2`, `function () use ($y)`
`this`	Phụ thuộc cách gọi hàm	`$this` luôn là object
Riêng tư	`#field`	`private`
Module	`import`, `export`, scope riêng	`use`, Composer autoload
Gói	`npm`, `package.json`	`composer`, `composer.json`
Mô hình chạy	Một process, event loop, bất đồng bộ	Mỗi request một worker, đồng bộ
Lỗi chưa bắt	Có thể làm sập cả server	Chỉ hỏng request đó

Lỗi hay gặp và cách đọc

Thông báo lỗi	Nguyên nhân thường gặp	Cách xử lý
`ReferenceError: x is not defined`	Chưa khai báo, sai tên, sai scope	Kiểm tra chính tả và scope

Thông báo lỗi	Nguyên nhân thường gặp	Cách xử lý
`ReferenceError: Cannot access 'x' before initialization`	Dùng `let` / `const` trong TDZ	Khai báo trước khi dùng
`TypeError: Cannot read properties of undefined`	Đọc thuộc tính của `undefined`: dữ liệu chưa về, hoặc mất `this`	Dùng `?.`, kiểm tra dữ liệu, sửa `this`
`TypeError: x is not a function`	Gọi giá trị không phải hàm, thiếu `export`, hoặc hoisting của `var`	In `typeof x`
`TypeError: Assignment to constant variable`	Gán lại `const`	Đổi sang `let`
`[object Object]` trong chuỗi	Ghép object vào chuỗi	`JSON.stringify(x)`
`[object Promise]`, `Promise { <pending> }`	Quên `await`	Thêm `await`
Dữ liệu `undefined` sau `then`	Quên `return` trong `then` hoặc arrow function có `{}`	Thêm `return`
`UnhandledPromiseRejection`, process thoát	Promise bị lỗi mà không có `catch`	Thêm `await` trong `try` hoặc `.catch()`
`ERR_MODULE_NOT_FOUND`	Thiếu đuôi file khi import ESM, sai đường dẫn	Viết đủ `./file.mjs`
`__dirname is not defined`	Dùng `__dirname` trong ESM	`import.meta.dirname`
`Cannot use import statement outside a module`	Dùng `import` trong file CommonJS	Đổi `.mjs`, hoặc đặt `"type": "module"`

Từ JS đến code React và NestJS

Thấy trong code	Thực chất là
`const [count, setCount] = useState(0)`	Destructuring mảng (chương 3), state trong closure (bài 9.1)
`function Card({ title })`	Destructuring tham số (bài 3.3)
`{jobs.map((j) => <Card key={j.id} />)}`	`map` trả mảng mới (chương 3)
`setJobs([...jobs, job])`	Tạo mảng mới để đổi tham chiếu (chương 2)
`setCount((c) => c + 1)`	Tránh stale closure (bài 4.6)
`useEffect(() => { ... return cleanup }, [id])`	Closure, so sánh dependency bằng `Object.is` (bài 9.2)
`React.lazy(() => import('./Page'))`	Dynamic import (chương 8)
`constructor(private readonly repo: Repo)`	Class và DI (chương 5, bài 9.6)
`async findOne(id) { return await this.repo... }`	Async/await (chương 6)

Thấy trong code	Thực chất là
`next.handle().pipe(map(...))`	Bọc quanh handler (bài 9.4), hàm bậc cao
`EventEmitter2`, `@OnEvent`	Pub/sub (bài 9.5), lưu ý tính đồng bộ
Queue BullMQ cho việc nặng	Tránh chặn event loop (chương 7)

Nên đọc thêm

- MDN JavaScript Guide: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
- Node.js, phần event loop: nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick
- Node.js, phần ES Module: nodejs.org/api/esm.html
- React, phần State as a Snapshot và Rules of Hooks: react.dev/learn
- Các file trong repo này: `react-crash-course/react-crash-course-plan.pdf` (Part J) và `nestjs-crash-course/nestjs-crash-course-plan.pdf` (Part P)